



RÉPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Université de Sousse
École Nationale d'Ingénieurs de Sousse



Projet de Fin d'Études

En vue de l'obtention du

Diplôme National d'Ingénieur En Génie Électronique Industrielle

Développement d'un système de gestion
de flotte par Jumeau Numérique
pour une flotte hétérogène de robots mobiles
autonomes

Réalisé par

Mohamed Amine Herch

Hébergé par « Centre National Intégré du Manufacturier Intelligent »



Soutenu le 16 septembre 2026, devant le jury composé de :

Mme. Asma Ben Rhouma	Présidente du jury
Mme. Hajer Gharsallaoui	Rapporteuse
M. Moncef Ghiss	Encadrant ENISO
Dr. Ali AMAMOU	Encadrant CNIMI

Année Universitaire : 2025 / 2026



RÉPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Université de Sousse
École Nationale d'Ingénieurs de Sousse



Projet de Fin d'Études

En vue de l'obtention du

Diplôme National d'Ingénieur En Génie Électronique Industrielle

Développement d'un système de gestion
de flotte par Jumeau Numérique
pour une flotte hétérogène de robots mobiles
autonomes

Réalisé par

Mohamed Amine Herch

Hébergé par « Centre National Intégré du Manufacturier Intelligent »



Encadrant CNIMI Dr. Ali AMAMOU	Encadrant ENISo M. Moncef Ghiss
Date :	Date :

Année Universitaire : 2025 / 2026

Remerciements

Je tiens à exprimer ma profonde reconnaissance à mon encadrant académique, **M. Moncef Ghiss**, dont le soutien, les conseils éclairés et l'encadrement précieux ont grandement façonné la qualité de ce travail ainsi que mon développement académique. Sa présence, non seulement en tant que professeur mais aussi en tant que modèle, n'a cessé d'inspirer et de faire avancer mon parcours.

Je tiens également à remercier sincèrement les membres du jury pour avoir consacré leur temps et leur savoir à l'évaluation de ce projet. J'espère sincèrement que les résultats de ce travail seront à la hauteur de leur considération et de leurs attentes.

Je souhaite exprimer ma profonde gratitude à **Dr. Ali Amamou**, mon encadrant au CNIMI, pour ses conseils précieux et ses encouragements motivants tout au long de la progression de ce projet. Mes remerciements s'adressent également à l'ensemble du personnel du CNIMI pour leur accueil et leur aide, et tout particulièrement à **Gervais-Lavois Jérôme**, **Ahmed Dhahri** et **Amine Romdhane**, dont l'aide et les conseils m'ont été d'une grande utilité durant ce projet.

Mes remerciements les plus profonds vont à mon **père**, dont les sacrifices et le soutien constant m'ont permis de réaliser mon rêve de devenir ingénieur. Je lui dois une gratitude que les mots ne sauraient exprimer.

Enfin, j'adresse mes remerciements les plus chaleureux à ma famille : à ma mère, à mes sœurs, ainsi qu'à mes amis les plus proches et à toutes les personnes qui m'ont soutenu tout au long de ce parcours.

Table des matières

Remerciements	i
Résumé	vii
Abstract	viii
Liste des acronymes	ix
Introduction générale	1
Chapitre 1 : Organisme d’accueil et cadre du projet	3
1.1 Organisme d’accueil	3
1.1.1 Activités	3
1.1.2 Espace vitrine technologique	4
1.2 Description de la plateforme robotique expérimentale	5
1.2.1 Plateforme mobile de manipulation : MiR250 équipé d’un bras UR10e	5
1.2.2 Plateforme AMR de manutention lourde : MiR600	6
1.2.3 Plateforme AMR à interface propriétaire : Omron LD-90	7
1.2.4 Analyse comparative et contraintes d’interopérabilité	8
1.3 Problématique	9
1.4 Objectifs du projet	10
1.4.1 Objectif général	10
1.4.2 Objectifs spécifiques	10
1.5 Méthodologie et planification	10
1.5.1 Méthodologie de travail	10
1.5.2 Planification	11
1.5.3 Jalons et indicateurs de performance	12
Conclusion	12
Chapitre 2 : Analyse bibliographique	14
Introduction	14
2.1 La Smart Factory	14
2.1.1 Technologies clés pour la coordination de ressources hétérogènes . .	15
2.2 Les robots mobiles autonomes	15
2.2.1 Définition et principe	15
2.2.2 Navigation autonome	16
2.2.3 Domaines d’application	17
2.3 La gestion de flotte robotique	17
2.3.1 Enjeux de la coordination multi-robots	17

2.3.2	Allocation et ordonnancement des tâches	17
2.3.3	Interopérabilité et standardisation	18
2.3.4	Frameworks de gestion de flotte et positionnement d'Open-RMF	18
2.3.5	Synthèse et choix retenu	19
2.4	Jumeau numérique pour la supervision et la gestion de flottes robotiques	19
2.4.1	Architecture et exigences d'un jumeau numérique de flotte	20
2.4.2	Synchronisation robot réel-modèle virtuel	20
2.4.3	Technologies de simulation et de visualisation	21
2.4.4	Applications et travaux existants en jumeau numérique de flotte	22
2.4.5	Positionnement de l'approche proposée	22
2.5	Synthèse et positionnement du projet	22
	Conclusion	23
Chapitre 3 : Conception et implémentation du système		25
	Introduction	25
3.1	Analyse des besoins	25
3.1.1	Besoins fonctionnels	25
3.1.2	Besoins non fonctionnels	26
3.2	Architecture globale de la solution	26
3.3	Couche d'intégration des robots	27
3.3.1	Driver des robots MiR	27
3.3.2	Driver de l'Omron LD-90	30
3.4	Orchestration de la flotte avec Open-RMF	34
3.4.1	Carte et graphe de navigation	34
3.4.2	Le Fleet Adapter	35
3.4.3	Cycle de vie d'une navigation	37
3.4.4	Calibration des repères robot-RMF	37
3.4.5	Le client de missions	38
3.4.6	Le Dispatcher et l'allocation des missions	38
3.4.7	Le Traffic Schedule et la négociation de trafic	39
3.4.8	Limites identifiées	39
3.5	Jumeau numérique sous Unity	40
3.5.1	Construction de l'environnement virtuel	41
3.5.2	Modélisation des robots (URDF)	42
3.5.3	Architecture de communication Unity-ROS 2	43
3.5.4	Mode Simple : supervision et contrôle par robot	44
3.5.5	Mode Flotte : intégration avec Open-RMF	45
3.5.6	Surveillance automatique de la batterie	46
3.5.7	Enregistrement de l'historique des missions	46
3.5.8	Gestion du système et intégration de bout en bout	47
	Conclusion	48
Chapitre 4 : Tests et validation expérimentale		50
	Introduction	50
4.1	Environnement et protocole de test	50
4.1.1	Environnement expérimental	50
4.1.2	Protocole expérimental	51
4.2	Validation en mode Simple	51
4.2.1	Validation des fonctions de commande	52

4.2.2	Validation de la supervision temps réel	54
4.2.3	Représentation de la flotte dans l'environnement virtuel	55
4.2.4	Analyse des résultats	56
4.2.5	Bilan de la validation en mode Simple	56
4.3	Validation en mode Flotte	57
4.3.1	Analyse de l'allocation automatique	59
4.3.2	Comparaison des temps d'exécution	59
4.3.3	Analyse du cas particulier de L50	60
4.3.4	Interprétation des résultats	60
4.3.5	Bilan de la validation du mode Flotte	61
4.4	Robustesse et essais complémentaires	61
4.4.1	Démarrage et arrêt répétés	62
4.4.2	Coordination de trafic	62
4.4.3	Coexistence des deux modes d'envoi de missions	63
4.4.4	Synchronisation en temps réel	63
4.5	Discussion	63
4.5.1	Pertinence de l'allocation des missions	64
4.5.2	Impact de l'orchestration	64
4.5.3	Apport de la coordination multi-robots	64
4.5.4	Limites de la campagne expérimentale	64
	Conclusion	65

Conclusion et perspectives	68
-----------------------------------	-----------

Table des figures

1.1	Bâtiment principal du CNIMI	3
1.2	Espace vitrine technologique	4
1.3	Robot mobile autonome MiR250 équipé d'un bras manipulateur UR10e . .	6
1.4	Robot mobile autonome MiR600	7
1.5	Robot mobile autonome Omron LD-90	8
1.6	Diagramme de Gantt du déroulement du projet	11
2.7	Comparaison entre un AGV à trajectoire fixe et un AMR à navigation dynamique	16
2.8	Distinction entre modèle, ombre et jumeau numériques [1]	21
3.1	Architecture globale de la solution et ses deux chaînes de contrôle	27
3.2	Architecture du driver MiR : canal WebSocket (temps réel) et canal REST (missions)	28
3.3	Architecture en couches du driver Omron LD-90 et flux bidirectionnel . . .	31
3.4	Architecture d'intégration d'Open-RMF (deux flottes) et chaîne d'exécution d'une mission	34
3.5	Annotation de la carte CNIMI et des voies de circulation dans Traffic Editor	35
3.6	Interface de commande et de supervision du jumeau numérique sous Unity	40
3.7	Environnement virtuel de la flotte reconstitué sous Unity (en haut) et la- boratoire CNIMI réel correspondant (en bas)	42
3.8	Modèles tridimensionnels des robots intégrés dans l'environnement virtuel : MiR250 équipé du bras UR10e, MiR600 et Omron LD-90	43
3.9	Architecture de communication entre Unity et ROS 2	44
4.1	Supervision simultanée des trois robots de la flotte dans l'environnement virtuel Unity	56
4.2	Comparaison des temps d'exécution selon le mode de fonctionnement . . .	59

Liste des tableaux

1.2	Principales caractéristiques du MiR250 et du bras UR10e	6
1.3	Principales caractéristiques du MiR600	7
1.4	Principales caractéristiques de l’Omron LD-90	8
1.5	Comparaison des robots de la flotte	9
1.6	Jalons de validation et indicateurs de performance	12
2.7	Comparaison des approches de gestion de flotte	19
2.8	Principales plateformes de développement de jumeaux numériques	22
3.1	Principales interfaces ROS 2 du nœud <code>mir_bridge</code>	28
3.2	Commandes gérées par le nœud <code>mir_mission_api</code>	29
3.3	Architecture logicielle du driver Omron LD-90	30
3.4	Principales interfaces ROS 2 du driver Omron LD-90 (namespace <code>/omron</code>)	33
3.5	Points d’intérêt du graphe de navigation, par catégorie	35
3.6	Configuration des robots des deux flottes	36
3.7	Méthodes du <i>RobotCommandHandle</i> et leur traduction	37
3.8	Décalages de calibration entre le repère de chaque robot et le repère RMF	38
3.9	Mode Simple : principales commandes envoyées par Unity	45
3.10	Mode Simple : principales données reçues par Unity	45
3.11	Champs enregistrés pour chaque mission	47
4.1	Environnement matériel et logiciel des essais	51
4.2	Fonctions évaluées lors de la validation du mode Simple	52
4.3	Synthèse des essais de commande réalisés en mode Simple	55
4.4	Robot sélectionné par Open-RMF et distances à vol d’oiseau aux destinations	58
4.5	Temps d’exécution en mode automatique et en mode manuel	58

Résumé

Ce projet de fin d'études porte sur la conception et l'implémentation d'un système de gestion de flotte destiné à un ensemble hétérogène de robots mobiles autonomes (AMR) au laboratoire CNIMI de l'Université du Québec à Trois-Rivières (UQTR), en vue de son intégration dans un jumeau numérique. La flotte étudiée regroupe des robots issus de différents fabricants, à savoir un MiR250 équipé d'un bras manipulateur UR10e, un MiR600 et un Omron LD-90. Chacun de ces robots dispose de sa propre pile de navigation propriétaire ainsi que de protocoles de communication distincts, notamment l'API REST pour les robots MiR et l'interface ARCL pour le robot Omron.

L'approche proposée repose sur le framework Open-RMF (*Open Robotics Middleware Framework*), utilisé comme couche d'orchestration unifiée et déployé sur ROS 2 Jazzy. Un adaptateur de flotte spécifique a été développé afin de traduire les commandes de planification et de gestion du trafic générées par Open-RMF vers les protocoles natifs de chaque robot, sans modifier leurs systèmes embarqués. Le système prend également en compte l'alignement des référentiels, intègre un client de missions modulaire ainsi qu'un pont de communication avec le jumeau numérique développé sous Unity, basé sur le ROS-TCP-Connector, afin d'assurer la visualisation, la supervision et la synchronisation en temps réel de l'ensemble de la flotte.

Les résultats obtenus valident le bon fonctionnement de l'architecture proposée : les robots reçoivent et exécutent physiquement des missions coordonnées via l'ordonnanceur de trafic d'Open-RMF. Ce travail démontre ainsi la faisabilité d'une gestion de flotte interopérable pour des robots de fabricants différents et pose les bases d'un jumeau numérique dédié à la supervision et à la simulation d'une flotte industrielle.

Mots-clés : robotique mobile, robots mobiles autonomes (AMR), gestion de flotte, flotte hétérogène, Open-RMF, ROS 2, jumeau numérique, orchestration du trafic, interopérabilité.

Abstract

This final-year project focuses on the design and implementation of a fleet management system for a heterogeneous fleet of Autonomous Mobile Robots (AMRs) at the CNIMI laboratory of the Université du Québec à Trois-Rivières (UQTR), with the objective of integrating it into a Digital Twin. The fleet consists of robots from different manufacturers, namely a MiR250 equipped with a UR10e manipulator arm, a MiR600, and an Omron LD-90. Each robot relies on its own proprietary navigation stack and communication protocol, including the REST API for MiR robots and the ARCL interface for the Omron robot.

The proposed approach is based on Open-RMF (*Open Robotics Middleware Framework*), which serves as a unified fleet orchestration layer deployed on ROS 2 Jazzy. A custom fleet adapter was developed to translate the traffic planning and fleet management commands generated by Open-RMF into each robot's native communication protocol without modifying their onboard software. The system also addresses coordinate frame alignment, integrates a modular mission client, and establishes a communication bridge with the Digital Twin developed in Unity using the ROS-TCP-Connector, enabling real-time visualization, supervision, and synchronization of the entire fleet.

The obtained results validate the proposed architecture by demonstrating that the robots successfully receive and execute coordinated missions through Open-RMF's traffic scheduler. This work demonstrates the feasibility of interoperable fleet management for heterogeneous mobile robots and establishes the foundations for a Digital Twin dedicated to the supervision and simulation of an industrial robotic fleet.

Keywords : mobile robotics, autonomous mobile robots (AMRs), fleet management, heterogeneous fleet, Open-RMF, ROS 2, Digital Twin, traffic orchestration, interoperability.

Liste des acronymes

2D	<i>Two-Dimensional</i>
3D	<i>Three-Dimensional</i>
AMR	<i>Autonomous Mobile Robot</i>
API	<i>Application Programming Interface</i>
ARCL	<i>Advanced Robotics Command Language</i>
CAD	<i>Computer-Aided Design</i>
CNIMI	Centre National Intégré du Manufacturier Intelligent
CSV	<i>Comma-Separated Values</i>
DM	<i>Digital Model</i>
DS	<i>Digital Shadow</i>
DT	<i>Digital Twin</i>
GUI	<i>Graphical User Interface</i>
HTTP	<i>HyperText Transfer Protocol</i>
IoT	<i>Internet of Things</i>
JSON	<i>JavaScript Object Notation</i>
LiDAR	<i>Light Detection and Ranging</i>
MiR	<i>Mobile Industrial Robots</i>
PC	<i>Personal Computer</i>
QoS	<i>Quality of Service</i>
REST	<i>Representational State Transfer</i>
RMF	<i>Robotics Middleware Framework</i>
ROS	<i>Robot Operating System</i>
SLAM	<i>Simultaneous Localization and Mapping</i>
TCP	<i>Transmission Control Protocol</i>
TF	<i>Transform</i> (arbre de transformations)
UI	<i>User Interface</i>
UQTR	Université du Québec à Trois-Rivières
UR	<i>Universal Robots</i>
URDF	<i>Unified Robot Description Format</i>
VR	<i>Virtual Reality</i>

Introduction générale

L'évolution rapide de l'automatisation industrielle à l'ère de l'Industrie 4.0 a favorisé l'intégration de systèmes de supervision avancés dans les environnements robotisés. Les milieux de production modernes exigent davantage de flexibilité, d'efficacité et de transparence, objectifs pouvant être atteints grâce à la combinaison des jumeaux numériques, des intergiciels robotiques et des robots mobiles autonomes. Ces technologies permettent la surveillance et le contrôle en temps réel, ainsi que l'analyse prédictive et la simulation virtuelle afin d'améliorer la fiabilité des opérations manufacturières.

Dans ce contexte, le jumeau numérique s'impose comme une technologie clé en offrant une représentation virtuelle des systèmes physiques synchronisée avec leur comportement réel. Il permet de visualiser, d'analyser et d'optimiser les procédés tout en réduisant les temps d'arrêt et en facilitant la prise de décision. Par ailleurs, le déploiement simultané de plusieurs robots mobiles autonomes (AMR) au sein d'un même site industriel soulève le défi de la coordination d'une flotte hétérogène composée de robots de constructeurs différents, utilisant des systèmes de navigation et des protocoles de communication propriétaires.

La mission du Centre National Intégré du Manufacturier Intelligent (CNIMI) est d'accompagner les entreprises manufacturières dans l'adoption des technologies de l'Industrie 4.0 afin d'améliorer leur compétitivité. Le projet est réalisé autour d'une flotte hétérogène composée d'un MiR250 équipé d'un bras manipulateur UR10e, d'un MiR600 et d'un Omron LD-90. Chaque robot disposant de son propre écosystème logiciel, leur supervision centralisée constitue un défi majeur.

Notre projet vise à concevoir et à implémenter un système de contrôle de flotte unifié, reposant sur le framework Open-RMF déployé sur ROS2 Jazzy, ainsi qu'un jumeau numérique développé sous Unity. Le système de contrôle de flotte assure l'orchestration du trafic, la planification et l'exécution des missions sur l'ensemble des robots, tandis que le jumeau numérique offre une représentation interactive et fidèle de la flotte, constituant un outil puissant pour la visualisation, la supervision et l'analyse des performances.

Ce rapport est organisé en quatre chapitres qui reflètent le déroulement de notre projet :

- **Le chapitre 1** présente l'organisme d'accueil, le cadre du projet, la problématique industrielle identifiée, les objectifs à atteindre ainsi que la méthodologie de travail adoptée.
- **Le chapitre 2** expose l'état de l'art, en mettant en évidence les principes de l'Industrie 4.0, des systèmes de gestion de flotte robotique et des jumeaux numériques, afin de situer et de justifier les choix technologiques du projet.
- **Le chapitre 3** détaille la conception et l'implémentation de la solution : la couche d'intégration des robots MiR et Omron, l'orchestration de la flotte à l'aide d'Open-RMF, ainsi que le développement du jumeau numérique de supervision sous Unity.
- **Le chapitre 4** présente la validation expérimentale du système sur la flotte réelle, à travers des scénarios de mission en modes Simple et Flotte, et analyse les résultats obtenus.

Chapitre 1 : Organisme d'accueil et cadre du projet

Chapitre 1 : Organisme d'accueil et cadre du projet

1.1 Organisme d'accueil

Situé à Drummondville, au Québec, le Centre National Intégré du Manufacturier Intelligent (CNIMI), illustré à la Figure 1.1, constitue une initiative stratégique visant à accompagner les entreprises québécoises dans leur transition vers l'Industrie 4.0.



FIGURE 1.1 – Bâtiment principal du CNIMI

Le centre a été créé à la suite d'une consultation menée en 2015 sur l'avenir économique de Drummondville. Il résulte d'une collaboration entre plusieurs acteurs du milieu éducatif, notamment l'Université du Québec à Trois-Rivières (UQTR) et le Cégep de Drummondville, ainsi que des partenaires socio-économiques tels que la Ville de Drummondville, la Société de développement économique de Drummondville et la Chambre de commerce et d'industrie de la région.

Cette collaboration repose sur une vision commune : favoriser le rapprochement entre la recherche académique, la formation et les besoins industriels afin d'accélérer l'adoption des technologies liées au manufacturier intelligent.

1.1.1 Activités

En tant que pôle de référence en manufacturier intelligent, le CNIMI accompagne les entreprises québécoises dans leur transformation numérique vers l'Industrie 4.0. Le

centre favorise la collaboration entre les industriels, les chercheurs et les spécialistes technologiques afin de proposer des solutions adaptées aux problématiques rencontrées sur le terrain.

Les principales activités proposées par le CNIMI comprennent :

- la réalisation d’audits Industrie 4.0 et l’élaboration de feuilles de route pour la transformation numérique ;
- l’accompagnement stratégique des entreprises à travers des diagnostics et des recommandations technologiques ;
- des services-conseils liés à l’intégration de technologies émergentes ;
- des programmes de formation et de coaching dans les domaines du manufacturier intelligent, de l’ingénierie et de la transformation numérique ;
- l’organisation de vitrines technologiques et de démonstrations industrielles ;
- des activités de réseautage et de collaboration entre les différents acteurs du domaine.

Parallèlement à ses activités d’accompagnement industriel, le CNIMI développe des projets de recherche appliquée portant notamment sur la numérisation tridimensionnelle, la robotique avancée et l’intelligence artificielle appliquée au manufacturier.

1.1.2 Espace vitrine technologique

Le CNIMI dispose d’un espace dédié à la vitrine technologique, présenté à la Figure 1.2. Cet environnement permet aux entreprises de découvrir les technologies associées à l’Industrie 4.0 à travers des démonstrations pratiques et des scénarios représentatifs des applications industrielles.

C’est dans cet environnement que s’inscrit le présent projet de fin d’études.



FIGURE 1.2 – Espace vitrine technologique

Cet espace regroupe plusieurs technologies innovantes, notamment :

- **Robots collaboratifs** : bras UR10e travaillant aux côtés des opérateurs.
- **Jumeaux numériques** : réplique virtuelle temps réel d’une ligne de production.
- **Robots mobiles autonomes** : MiR250, MiR600 et Omron LD-90 pour la logistique interne.
- **Réalité étendue (VR/AR/MR)** : casques immersifs pour la formation et la maintenance assistée.
- **Instructions de travail numériques** : guides opératoires interactifs sur écran ou tablette.

- **Intelligence artificielle** : inspection qualité par vision et maintenance prédictive.
- **Exosquelettes** : assistance physique lors du port de charges lourdes.
- **Interconnexion des systèmes** : protocoles OPC UA et MQTT reliant machines et logiciels.
- **Solutions logicielles industrielles** : systèmes MES et SCADA de supervision de production.
- **Localisation et traçabilité** : systèmes de positionnement intérieur (IPS) et RFID.
- **Fabrication additive** : impression 3D de pièces et de prototypes.
- **Collecte de données et Internet des objets (IoT)** : capteurs connectés remontant les données machines.

1.2 Description de la plateforme robotique expérimentale

La flotte mise à notre disposition au CNIMI est de nature hétérogène : elle réunit des robots mobiles autonomes (AMR) issus de constructeurs distincts, chacun reposant sur son propre système de navigation et sur des protocoles de communication qui lui sont propres. Cette diversité, si elle enrichit les possibilités d’usage, constitue également le principal défi d’intégration abordé dans ce projet. Nous décrivons ci-dessous chaque plateforme, ses caractéristiques techniques ainsi que le moyen de communication qu’elle expose. La structure adoptée est volontairement modulaire, de sorte que l’ajout ultérieur d’un nouveau robot à la flotte se traduise simplement par l’ajout d’une nouvelle sous-section.

1.2.1 Plateforme mobile de manipulation : MiR250 équipé d’un bras UR10e

Le MiR250 est un robot mobile autonome développé par Mobile Industrial Robots (MiR), destiné au transport autonome de charges dans des environnements industriels [2]. Avec une charge utile pouvant atteindre 250 kg et une vitesse maximale de 2 m/s, il est conçu pour circuler dans des espaces industriels restreints tout en assurant une navigation autonome.

Sa navigation repose sur une carte de l’environnement et un système de localisation autonome exploitant plusieurs capteurs, notamment des scrutateurs laser de sécurité et des caméras 3D. Ces capteurs permettent au robot de percevoir son environnement, de détecter les obstacles et d’assurer une navigation sécurisée [2].

Le MiR250 disponible au CNIMI est équipé d’un bras manipulateur collaboratif UR10e développé par Universal Robots, formant ainsi une plateforme mobile manipulatrice. Le bras possède six degrés de liberté, une charge utile de 12,5 kg, une portée de 1300 mm et une répétabilité de $\pm 0,05$ mm [3].

Dans le cadre de ce projet, le MiR250 constitue l’une des plateformes intégrées au système de gestion de flotte développé avec Open-RMF. La communication avec le robot repose sur une API REST permettant de récupérer ses informations d’état et de lui transmettre des missions depuis un système externe.



FIGURE 1.3 – Robot mobile autonome MiR250 équipé d'un bras manipulateur UR10e

Le MiR250 permet également le déplacement autonome de palettes et de charges lourdes. Le robot disponible au CNIMI est équipé d'un module de levage de palettes (*Pallet Lift*), permettant la manipulation autonome de palettes industrielles.

TABLE 1.2 – Principales caractéristiques du MiR250 et du bras UR10e

Caractéristique	Valeur
Charge utile du MiR250	250 kg
Vitesse maximale	2 m/s
Navigation	Cartographie et localisation autonomes (laser, caméras 3D)
Interface de communication	API REST
Charge utile de l'UR10e	12,5 kg
Portée du bras	1300 mm
Répétabilité	$\pm 0,05$ mm

1.2.2 Plateforme AMR de manutention lourde : MiR600

Le deuxième robot de la flotte est un MiR600, conçu par Mobile Industrial Robots pour des applications nécessitant le transport de charges plus importantes [4]. Avec une capacité de charge pouvant atteindre 600 kg et une vitesse maximale de 2 m/s, il est adapté au déplacement autonome de palettes et de charges lourdes. Le robot disponible au CNIMI est équipé d'un module de levage de palettes (*Pallet Lift*), permettant la manipulation autonome de palettes industrielles.



FIGURE 1.4 – Robot mobile autonome MiR600

Comme le MiR250, le MiR600 utilise une architecture logicielle propriétaire et expose des interfaces permettant son intégration avec des systèmes externes, notamment une API REST ainsi que des protocoles industriels tels que Modbus TCP et OPC UA. Il est intégré à la même architecture que le MiR250 afin de permettre la coordination de missions avec les autres robots de la flotte.

TABLE 1.3 – Principales caractéristiques du MiR600

Caractéristique	Valeur
Charge utile	600 kg
Vitesse maximale	2 m/s
Navigation	Cartographie et localisation autonomes (laser, caméras 3D)
Module	Levage de palettes (<i>Pallet Lift</i>)
Protocoles de communication	API REST, Modbus TCP, OPC UA

1.2.3 Plateforme AMR à interface propriétaire : Omron LD-90

Le troisième robot de la flotte est un Omron LD-90, un AMR conçu par Omron pour le transport de charges légères dans les environnements industriels [5]. Il supporte une charge utile allant jusqu'à 90 kg et atteint une vitesse maximale de 1,35 m/s.

Contrairement aux systèmes nécessitant des infrastructures physiques telles que des bandes magnétiques ou des marqueurs, l'Omron LD-90 utilise une navigation autonome basée sur la cartographie et la localisation de son environnement. Le robot dispose de plusieurs dispositifs de sécurité, notamment des capteurs laser, un sonar arrière et des systèmes de détection d'obstacles.



FIGURE 1.5 – Robot mobile autonome Omron LD-90

La communication avec l’Omron LD-90 repose sur le langage propriétaire ARCL (*Advanced Robotics Command Language*). Ce protocole textuel basé sur TCP permet l’envoi de commandes, la récupération d’informations d’état et la supervision du robot depuis une application externe. Dans ce projet, l’Omron LD-90 permet de valider l’interopérabilité de la solution proposée entre des plateformes de fabricants différents, son interface ARCL constituant le point d’intégration utilisé par l’adaptateur de flotte pour dialoguer avec Open-RMF.

TABLE 1.4 – Principales caractéristiques de l’Omron LD-90

Caractéristique	Valeur
Charge utile	90 kg
Vitesse maximale	1,35 m/s
Navigation	Cartographie et localisation autonomes (sans infrastructure)
Sécurité	Capteurs laser, sonar arrière, détection d’obstacles
Protocoles de communication	ARCL (<i>Advanced Robotics Command Language</i>), TCP

1.2.4 Analyse comparative et contraintes d’interopérabilité

Les trois plateformes retenues couvrent des fonctions complémentaires au sein de la flotte : la manipulation mobile (MiR250-UR10e), la manutention de charges lourdes (MiR600) et le transport léger à interface propriétaire (Omron LD-90). Le Tableau 1.5 en résume les principales caractéristiques.

TABLE 1.5 – Comparaison des robots de la flotte

Robot	Charge utile	Vitesse	Communication	Particularité
MiR250	250 kg	2 m/s	REST	Bras UR10e
MiR600	600 kg	2 m/s	REST	Levage de palettes
Omron LD-90	90 kg	1,35 m/s	ARCL	Navigation sans infrastructure

Au-delà de leurs différences de charge utile ou de vitesse, ces plateformes se distinguent surtout par leurs **interfaces de communication** et leurs **piles de navigation**, propriétaires et mutuellement incompatibles. Les robots MiR exposent une **API REST** sur HTTP, tandis que l’Omron LD-90 s’appuie sur le protocole textuel **ARCL** sur TCP. Chaque constructeur impose en outre son propre référentiel de localisation, son format de missions et son système de gestion embarqué.

Cette hétérogénéité soulève trois contraintes d’interopérabilité qui structurent la suite du travail :

- **Contrainte de communication** : l’absence d’interface commune impose une couche d’abstraction traduisant un modèle de commande unifié vers les protocoles natifs de chaque robot (REST pour MiR, ARCL pour Omron).
- **Contrainte spatiale** : chaque robot évoluant dans son propre référentiel de localisation, la coordination exige l’établissement d’un référentiel spatial commun à l’ensemble de la flotte.
- **Contrainte d’orchestration** : aucune plateforme n’offre nativement l’allocation de missions ni la coordination du trafic entre robots de constructeurs différents, ce qui nécessite une couche d’orchestration externe.

Ces trois contraintes (communication, spatiale et orchestration) définissent le problème d’ingénierie traité dans ce projet et appellent la mise en place d’une couche d’intégration logicielle et d’une couche d’orchestration de flotte, dont la conception fait l’objet des chapitres suivants.

1.3 Problématique

L’analyse comparative précédente met en évidence une flotte dont les plateformes reposent sur des protocoles de commande propriétaires et incompatibles, des référentiels de localisation distincts et des systèmes de gestion embarqués indépendants. Chaque famille de robots expose en effet sa propre interface : les robots MiR se commandent via une API REST et un flux WebSocket, tandis que l’Omron LD-90 s’appuie sur le protocole textuel ARCL transporté sur TCP. Ces interfaces reposent sur des formats de commande, des unités et des référentiels différents, sans langage commun.

En l’absence de couche d’orchestration commune, ces robots fonctionnent en silos : ils ne partagent ni référentiel spatial commun, ni système d’allocation de missions, ni mécanisme de coordination de trafic. Il en résulte l’impossibilité de les faire coopérer dans un même espace de travail sans risque de conflit, ainsi que l’absence d’une supervision unifiée de la flotte.

La problématique de ce travail peut alors être formulée ainsi : *comment concevoir une couche logicielle d’interopérabilité permettant d’orchestrer, de coordonner et de superviser, au sein d’un environnement partagé, une flotte hétérogène de robots mobiles dont les systèmes de navigation et les protocoles de communication sont propriétaires et incompatibles*

(API REST et WebSocket pour les robots MiR, protocole ARCL sur TCP pour l'Omron LD-90), tout en offrant une représentation temps réel de cette flotte à travers un jumeau numérique ?

1.4 Objectifs du projet

1.4.1 Objectif général

En réponse à cette problématique, ce projet a pour objectif général de *concevoir, implémenter et valider une architecture interopérable de gestion et de supervision d'une flotte hétérogène de robots mobiles autonomes*, reposant sur une couche d'orchestration commune (Open-RMF) et un jumeau numérique de supervision développé sous Unity.

1.4.2 Objectifs spécifiques

L'objectif général du projet se décline en cinq objectifs spécifiques (OS), définis de manière mesurable et directement associés aux principales contributions réalisées dans le cadre de ce travail :

- OS1** Concevoir une couche d'intégration logicielle sous ROS 2 permettant d'interfacer des robots mobiles hétérogènes avec une architecture commune, en adaptant leurs interfaces de communication propriétaires, notamment l'API REST pour les robots MiR et le protocole ARCL pour l'Omron LD-90.
- OS2** Mettre en place un référentiel spatial commun permettant l'intégration des différents robots dans un même environnement, et déterminer les transformations nécessaires entre les référentiels propres à chaque plateforme et le référentiel global de la flotte.
- OS3** Intégrer et configurer le framework Open-RMF comme couche d'orchestration de la flotte afin de permettre la planification, l'allocation et la coordination des missions ainsi que la gestion du trafic entre les robots provenant de constructeurs différents.
- OS4** Développer un client de missions modulaire permettant de soumettre et de superviser les missions des différents robots, puis valider expérimentalement leur exécution en configuration mono-robot et multi-robots, notamment à travers des scénarios de coordination et de gestion des conflits de trafic.
- OS5** Concevoir un jumeau numérique sous Unity permettant de représenter et de superviser la flotte physique, avec une synchronisation des données d'état des robots via ROS 2 et le ROS-TCP-Connector, afin d'assurer une visualisation cohérente de l'environnement et des plateformes robotiques.

La validation de ces objectifs repose sur des critères et des indicateurs de performance définis dans le cadre de la planification du projet, présentés dans le Tableau 1.6.

1.5 Méthodologie et planification

1.5.1 Méthodologie de travail

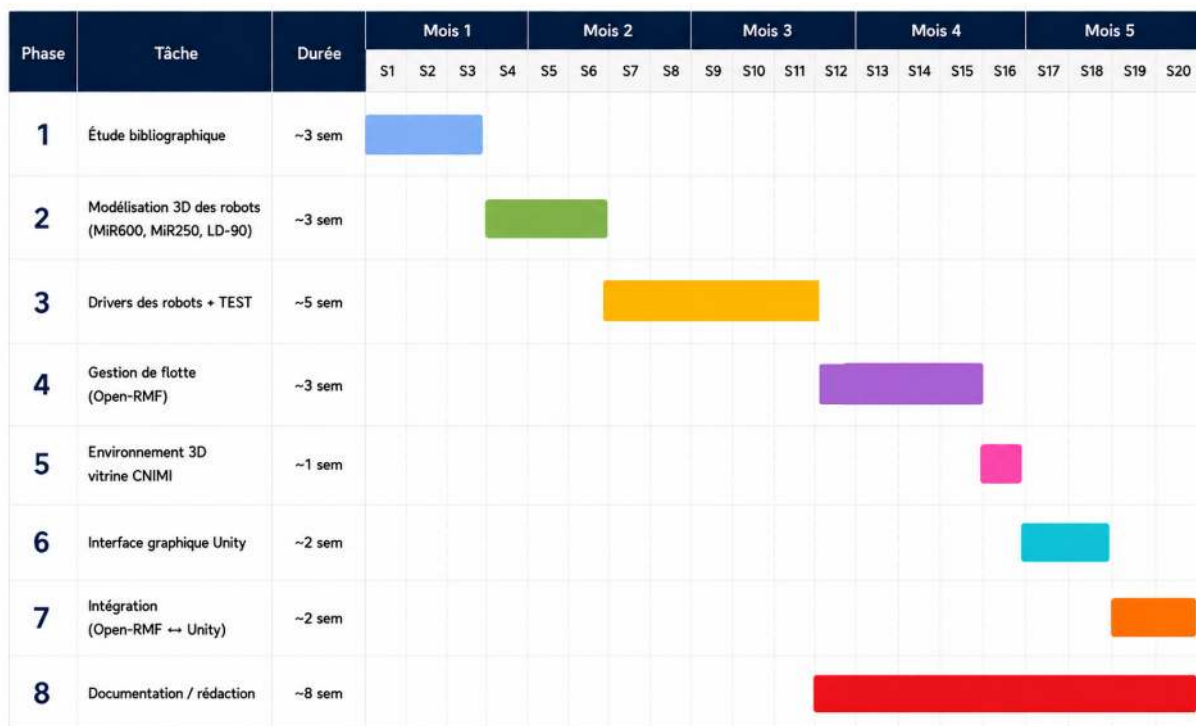
Le déroulement de ce projet a suivi une démarche structurée et itérative, garantissant clarté et traçabilité. Des échanges réguliers avec les encadrants ont permis de suivre l'avancement, d'analyser les difficultés rencontrées et d'orienter les choix techniques.

Le travail s'est appuyé sur les principes suivants :

- **Étude bibliographique** : une phase initiale de recherche a été consacrée à l'état de l'art des technologies de gestion de flotte de robots mobiles autonomes (AMR), du framework Open-RMF et des jumeaux numériques, afin de bien cerner les concepts avant l'implémentation.
- **Développement itératif** : le développement a suivi une approche incrémentale basée sur l'intégration progressive des différents composants logiciels : communication avec les robots, développement des adaptateurs de flotte, orchestration via Open-RMF et synchronisation avec le jumeau numérique.
- **Tests progressifs** : chaque fonctionnalité a été validée progressivement dans un environnement contrôlé avant son déploiement sur les plateformes robotiques réelles.
- **Documentation continue** : l'avancement, les problèmes rencontrés et les solutions apportées ont été consignés régulièrement, facilitant le suivi du projet et la rédaction du présent rapport.

1.5.2 Planification

Afin d'organiser le travail dans le temps et de respecter les échéances, une planification a été établie dès le début du projet. Le diagramme de Gantt de la Figure 1.6 présente les principales étapes et leur répartition sur la durée du stage.



S1 à S20 : semaines du projet (20 semaines au total, soit 5 mois).

FIGURE 1.6 – Diagramme de Gantt du déroulement du projet

1.5.3 Jalons et indicateurs de performance

Au-delà des tâches et de leur durée présentées dans le diagramme de Gantt, des jalons de validation ont été définis afin de suivre l'atteinte des différents objectifs spécifiques. Le Tableau 1.6 présente les principaux jalons ainsi que les indicateurs de performance (KPI) retenus pour évaluer les fonctionnalités développées.

TABLE 1.6 – Jalons de validation et indicateurs de performance

Jalon	OS	Critère de validation (KPI)
Intégration des robots MiR validée	OS1	Communication établie avec les MiR250 et MiR600 via l'API REST et exécution d'une mission depuis la couche d'intégration ROS 2.
Intégration de l'Omron validée	OS1	Communication établie avec l'Omron LD-90 via le protocole ARCL et exécution d'une mission depuis la couche d'intégration ROS 2.
Référentiel spatial commun établi	OS2	Transformation des coordonnées entre les référentiels des différents robots et le référentiel global, avec vérification expérimentale de la cohérence des positions dans l'environnement virtuel.
Orchestration Open-RMF opérationnelle	OS3	Mettre en place une architecture d'orchestration basée sur Open-RMF permettant de gérer les missions et les déplacements des différentes plateformes au sein d'un environnement commun.
Coordination multi-robots validée	OS4	Permettre la création, l'envoi et l'allocation automatique des missions aux plateformes disponibles, ainsi que la coordination de leur circulation.
Synchronisation avec le jumeau numérique validée	OS5	Mise à jour des états et des positions des robots dans Unity à partir des données transmises par ROS 2 via le ROS-TCP-Connector.

Conclusion

Ce premier chapitre a permis de situer le projet dans son contexte, en présentant l'organisme d'accueil, le CNIMI, ainsi que sa vitrine technologique. Nous avons ensuite décrit la flotte robotique hétérogène étudiée et détaillé les caractéristiques de chacune de ses plateformes. La problématique liée à l'absence d'une couche d'interopérabilité permettant la coordination de robots provenant de constructeurs différents a été exposée, puis traduite en un cahier des charges précis. Enfin, la méthodologie et la planification adoptées ont été présentées. Le chapitre suivant sera consacré à l'état de l'art des technologies mobilisées dans ce projet.

Chapitre 2 : Analyse bibliographique

Chapitre 2 : Analyse bibliographique

Introduction

Ce deuxième chapitre a pour objectif de situer le projet dans son cadre scientifique et technologique. Nous présentons d’abord le contexte de l’Industrie 4.0, dont s’inspirent les solutions développées. Nous nous intéressons ensuite aux robots mobiles autonomes et aux principes qui régissent leur navigation. Nous abordons par la suite la question centrale de la gestion de flotte robotique, en introduisant l’intergiciel ROS 2 ainsi que le framework Open-RMF. Enfin, nous examinons le concept de jumeau numérique et ses différentes déclinaisons. Cet état de l’art constitue le socle théorique sur lequel reposent les choix de conception présentés dans les chapitres suivants.

2.1 La Smart Factory

L’Industrie 4.0 marque une évolution des systèmes de production vers des environnements davantage connectés, intelligents et capables d’exploiter les données issues des équipements en temps réel. Cette transformation repose notamment sur l’intégration des technologies numériques, des systèmes cyber-physiques, de l’Internet des objets et de l’intelligence artificielle afin d’améliorer la flexibilité, la supervision et l’efficacité des processus industriels [6].

Parmi les fondements de cette transformation, les systèmes cyber-physiques (*Cyber-Physical Systems*, CPS) occupent une place centrale. Ils associent des éléments physiques, tels que les capteurs, les actionneurs et les systèmes embarqués, à des composants logiciels capables de traiter et d’échanger les données issues du système réel. La communication entre ces composantes établit un lien continu entre le processus physique et les services numériques qui lui sont associés. Dans le contexte de la robotique mobile, cette approche rend possible la collecte des informations relatives à l’état, à la position et aux activités des robots afin de les exploiter dans des systèmes de supervision et de décision, comme la couche ROS 2 développée dans ce projet pour recueillir l’état des robots MiR et Omron. Cette interaction entre le système physique et sa représentation numérique constitue également une base importante pour la mise en place d’un jumeau numérique, dans lequel les données provenant des robots physiques alimentent leur représentation virtuelle et facilitent leur supervision.

Ces principes trouvent une application concrète dans la *Smart Factory*, ou usine intelligente. Celle-ci repose sur l’interconnexion des équipements industriels, l’échange de données et l’utilisation de systèmes numériques pour améliorer la surveillance et la gestion des opérations de production. Dans un tel environnement, les robots mobiles autonomes assurent le transport de matériaux entre différentes zones, tandis que les systèmes de supervision suivent leur activité et leur état, et que les jumeaux numériques en offrent une représentation virtuelle. La vitrine technologique du CNIMI constitue précisément un environnement représentatif de cette approche : la présence de plusieurs plateformes robotiques hétérogènes y offre un contexte concret pour étudier leur intégration au sein

d’une architecture commune de gestion de flotte et pour développer une représentation numérique destinée à leur supervision.

La robotique mobile autonome occupe ainsi une place importante dans l’automatisation des opérations de manutention et de transport interne. L’utilisation simultanée de robots provenant de fabricants différents introduit toutefois des difficultés d’intégration, puisque chaque plateforme dispose de sa propre architecture logicielle, de ses mécanismes de localisation et de ses interfaces de communication. La mise en place d’une architecture capable d’assurer l’interopérabilité entre ces équipements constitue donc un enjeu majeur, que les technologies présentées dans la section suivante permettent d’adresser.

2.1.1 Technologies clés pour la coordination de ressources hétérogènes

L’automatisation des environnements industriels modernes repose sur l’association de plusieurs technologies complémentaires. Dans le cadre de ce projet, deux composantes occupent une place centrale : les robots mobiles autonomes (AMR) et les jumeaux numériques.

Les AMR assurent l’automatisation du déplacement de matériaux et de charges dans les environnements industriels tout en adaptant leurs trajectoires aux conditions rencontrées. Lorsqu’une flotte regroupe plusieurs plateformes provenant de constructeurs différents, leur coordination devient cependant plus complexe en raison de la diversité des interfaces et des architectures logicielles utilisées.

Les jumeaux numériques apportent quant à eux une représentation virtuelle du système physique, offrant la possibilité de visualiser son état et d’exploiter les données issues de son fonctionnement. Leur association avec une flotte d’AMR ouvre la voie à une supervision centralisée des robots et de leur environnement.

Pour assurer l’interaction entre ces différentes composantes, une couche logicielle d’interopérabilité est nécessaire. Elle doit notamment abstraire les spécificités des plateformes robotiques et fournir aux services de niveau supérieur une représentation commune des robots. Cette problématique motive l’utilisation d’une architecture fondée sur ROS 2 et Open-RMF¹ dans le présent projet.

2.2 Les robots mobiles autonomes

2.2.1 Définition et principe

L’automatisation du transport interne dans les environnements industriels a longtemps reposé sur les véhicules à guidage automatique (AGV), qui suivent des trajectoires prédéfinies matérialisées par des bandes magnétiques, des rails ou des marqueurs au sol. Si cette approche reste fiable pour des flux fixes et répétitifs, elle manque de souplesse dès que l’environnement évolue ou qu’un obstacle imprévu survient.

C’est pour dépasser ces limites qu’ont émergé les robots mobiles autonomes (AMR, *Autonomous Mobile Robots*). Plutôt que de suivre un chemin imposé, ces robots perçoivent leur environnement à l’aide de capteurs et déterminent leur trajectoire en temps réel [7],

1. Open-RMF (*Open Robotics Middleware Framework*) est un ensemble de bibliothèques open source, bâti sur ROS 2 et géré depuis 2024 par l’Open Source Robotics Alliance, dédié à l’orchestration des missions et à la coordination du trafic de flottes robotiques hétérogènes. Projet : <https://www.open-rmf.org/> — dépôt : <https://github.com/open-rmf/rmf>.

ce qui leur permet de se déplacer de façon autonome sans recourir à une infrastructure de guidage fixe.

La Figure 2.7 illustre cette différence fondamentale. Un AGV suit un chemin fixe : lorsqu'un obstacle bloque sa route, il s'arrête et ne peut poursuivre sa mission tant que la voie n'est pas dégagée. À l'inverse, un AMR s'appuie sur ses capteurs, tels que le LiDAR, pour percevoir son environnement, recalculer sa trajectoire en temps réel et contourner l'obstacle. Cette autonomie confère aux AMR une grande flexibilité : ils peuvent emprunter des itinéraires alternatifs et s'adapter en continu aux modifications de leur environnement de travail.

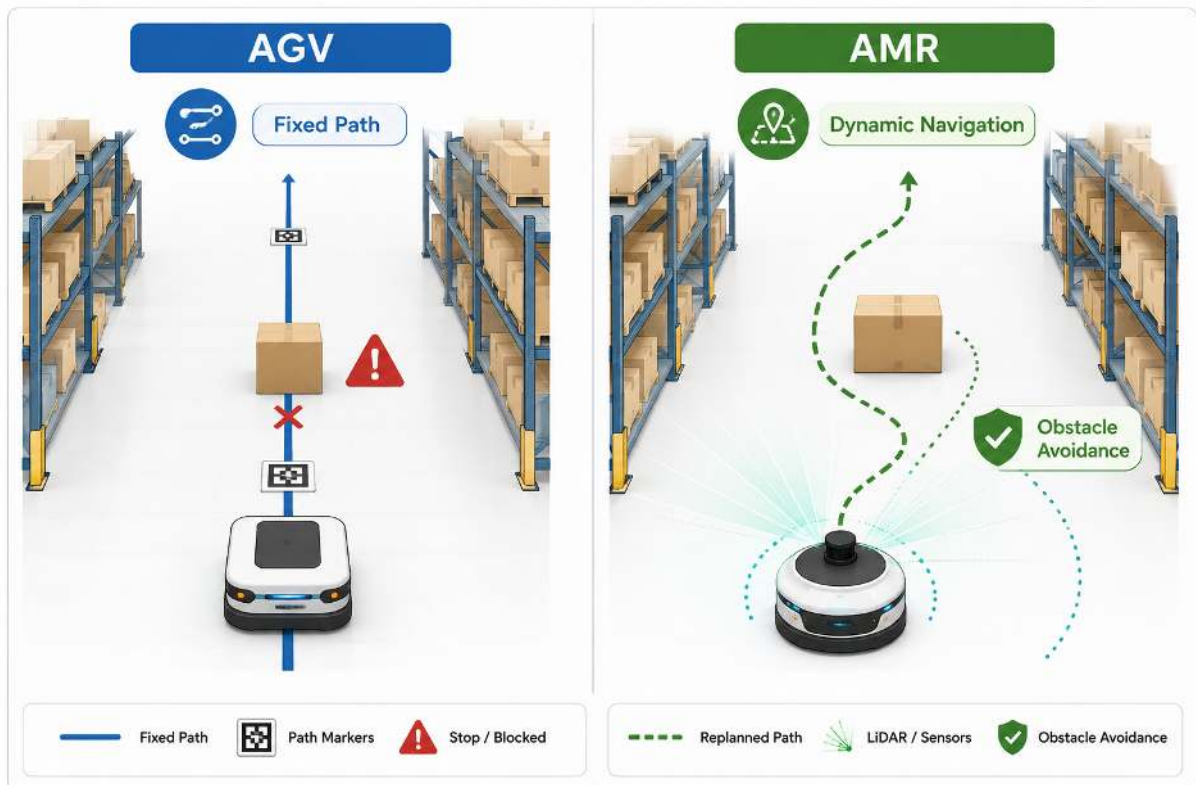


FIGURE 2.7 – Comparaison entre un AGV à trajectoire fixe et un AMR à navigation dynamique

2.2.2 Navigation autonome

La navigation d'un robot mobile autonome repose sur trois fonctions fondamentales et complémentaires :

- **La cartographie et la localisation** : le robot construit une carte de son environnement et détermine sa propre position à l'intérieur de celle-ci. Cette double tâche est généralement réalisée simultanément grâce à la technique de localisation et cartographie simultanées (SLAM, *Simultaneous Localization and Mapping*), qui s'appuie sur les données de capteurs tels que les scrutateurs laser et les caméras.
- **La planification de trajectoire** : à partir de la carte et de sa position, le robot calcule un chemin vers sa destination, en tenant compte des zones interdites et de la configuration de l'espace.
- **L'évitement d'obstacles** : en cours de déplacement, le robot détecte les obstacles dynamiques (personnes, autres robots, objets) et ajuste localement sa trajectoire

afin de garantir un déplacement sûr.

Cette combinaison permet aux AMR d'évoluer de manière fiable dans des environnements partagés avec des humains et d'autres véhicules.

2.2.3 Domaines d'application

Grâce à leur flexibilité, les robots mobiles autonomes sont aujourd'hui déployés dans de nombreux secteurs. Dans l'industrie manufacturière et la logistique, ils assurent le transport de composants, de palettes et de marchandises entre les postes de travail, réduisant ainsi les manutentions manuelles. Ils trouvent également des applications dans des environnements plus exigeants, comme le milieu hospitalier, où des flottes d'AMR sont utilisées pour transporter des échantillons ou du matériel médical dans des couloirs fréquentés [8]. Ces déploiements montrent que l'intérêt des AMR ne réside pas uniquement dans leurs capacités individuelles, mais surtout dans leur exploitation collective au sein d'une flotte, ce qui soulève la question de leur coordination, abordée dans la section suivante.

2.3 La gestion de flotte robotique

Après avoir présenté les robots mobiles autonomes, cette section s'intéresse à la manière dont plusieurs d'entre eux peuvent être exploités conjointement au sein d'une flotte. Elle aborde successivement les enjeux de la coordination multi-robots, l'allocation et l'ordonnancement des tâches, la question de l'interopérabilité, puis les principaux frameworks de gestion de flotte, afin de situer le choix d'Open-RMF retenu dans ce projet.

2.3.1 Enjeux de la coordination multi-robots

Lorsque plusieurs robots mobiles autonomes évoluent dans un même espace, leur exploitation individuelle ne suffit plus : il devient nécessaire de les coordonner au sein d'une flotte. La gestion de flotte désigne l'ensemble des mécanismes permettant de superviser, de planifier et de coordonner les déplacements et les tâches d'un groupe de robots afin d'atteindre un objectif global. Cette coordination soulève plusieurs enjeux : répartir efficacement les tâches entre les robots, éviter les conflits de trajectoire lorsqu'ils partagent les mêmes voies de circulation, et garantir un fonctionnement fluide et sûr de l'ensemble. La difficulté s'accroît lorsque la flotte est hétérogène, c'est-à-dire composée de robots aux capacités et aux caractéristiques différentes [8].

2.3.2 Allocation et ordonnancement des tâches

Un aspect central de la gestion de flotte concerne l'allocation des tâches, qui consiste à décider quel robot doit exécuter quelle mission, et dans quel ordre. Ce problème devient particulièrement complexe dans le cas de flottes hétérogènes, où les robots ne possèdent pas tous les mêmes aptitudes. Différentes approches ont été proposées dans la littérature, allant des méthodes d'optimisation classiques aux techniques d'apprentissage automatique. Récemment, des travaux ont exploré l'utilisation de l'apprentissage par renforcement pour l'allocation et l'ordonnancement de tâches au sein de flottes multi-robots hétérogènes, offrant des solutions capables de s'adapter dynamiquement à l'évolution de l'environnement et de la charge de travail [9].

2.3.3 Interopérabilité et standardisation

L'un des principaux obstacles à la gestion d'une flotte hétérogène réside dans la diversité des interfaces et des protocoles propres à chaque constructeur. Pour répondre à ce problème, des efforts de standardisation ont vu le jour, notamment avec la norme VDA5050, qui définit une interface de communication unifiée entre les robots mobiles et un système de gestion de flotte central. Cette norme vise à permettre l'exploitation conjointe de robots provenant de fabricants différents au sein d'une même installation [10]. La standardisation constitue ainsi une étape clé vers des systèmes réellement interopérables, capables d'intégrer des plateformes variées sans dépendre d'un unique fournisseur.

2.3.4 Frameworks de gestion de flotte et positionnement d'Open-RMF

Plusieurs stratégies peuvent être envisagées pour assurer la gestion et la coordination d'une flotte de robots mobiles autonomes. Leur pertinence dépend notamment du degré d'hétérogénéité des plateformes, de leurs interfaces de communication ainsi que des besoins en matière de planification et de coordination du trafic. Dans le cadre de ce projet, trois grandes approches peuvent être distinguées.

Solutions propriétaires par constructeur : Les constructeurs de robots mobiles proposent généralement leurs propres systèmes de gestion de flotte. Ces solutions sont conçues pour exploiter étroitement les fonctionnalités et les interfaces des plateformes du même constructeur. Elles présentent ainsi l'avantage d'une intégration directe et d'un environnement logiciel maîtrisé. En revanche, leur utilisation dans une flotte composée de robots provenant de fabricants différents nécessite des mécanismes d'intégration supplémentaires. Elles ne constituent donc pas, à elles seules, une solution générique adaptée à une flotte hétérogène telle que celle étudiée dans ce projet.

Développements spécifiques et coordination par zones : Une deuxième approche consiste à développer une solution de coordination spécifique au besoin considéré. Celle-ci peut, par exemple, s'appuyer sur des mécanismes de réservation de zones, de priorité ou de verrous d'exclusion mutuelle (*mutex*) afin de contrôler l'accès des robots aux zones partagées. Cette solution peut être suffisante pour des scénarios simples et un nombre limité de robots. Cependant, lorsque la flotte évolue, son extension nécessite le développement et la maintenance d'interfaces spécifiques pour les différentes plateformes, et la complexité de la coordination du trafic augmente avec le nombre de robots et de zones partagées.

Frameworks ouverts d'orchestration : Une troisième approche repose sur l'utilisation de frameworks ouverts destinés à l'orchestration de flottes hétérogènes. Ces solutions cherchent à séparer les fonctions de coordination de flotte des spécificités propres à chaque constructeur. La standardisation VDA5050, présentée précédemment, constitue une approche visant à unifier les échanges entre un système de contrôle et différents robots mobiles. Elle définit toutefois un cadre de communication et ne constitue pas, à elle seule, un système complet de planification et de coordination du trafic.

Dans cette catégorie, Open-RMF (*Open Robotics Middleware Framework*) propose une architecture d'orchestration fondée sur ROS 2. Celle-ci intègre différentes plateformes

robotiques au moyen de *fleet adapters*, chargés d’assurer l’interface entre les mécanismes d’orchestration et les systèmes propres à chaque flotte. Open-RMF fournit également des mécanismes de planification des déplacements et de coordination du trafic dans un environnement partagé [8, 11].

2.3.5 Synthèse et choix retenu

Le Tableau 2.7 compare ces trois approches au regard des contraintes d’interopérabilité identifiées au chapitre précédent : la contrainte de communication (support multi-constructeurs), la contrainte spatiale et d’orchestration (planification et coordination du trafic), le passage à l’échelle (*scalabilité*) lorsque le nombre de robots augmente, ainsi que le coût d’intégration et de maintenance. Les solutions propriétaires offrent une intégration étroite avec leur écosystème mais restent limitées à un seul constructeur ; les développements spécifiques offrent une grande liberté d’adaptation mais leur maintenance et leur passage à l’échelle deviennent complexes lorsque la flotte s’agrandit. Les frameworks ouverts d’orchestration répondent quant à eux simultanément à ces différentes contraintes.

TABLE 2.7 – Comparaison des approches de gestion de flotte

Approche	Multi-constructeurs	Planification / trafic	Scalabilité	Coût d’intégration
Solutions propriétaires	Non	Oui (intra-marque)	Limitée	Faible (mono-marque)
Développements spécifiques / zones-mutex	Oui	Limité	Faible	Élevé
Frameworks ouverts (Open-RMF)	Oui	Oui	Élevée	Modéré

Dans le cadre de ce projet, Open-RMF a été retenu comme couche d’orchestration de la flotte. Ce choix est motivé par son intégration avec ROS 2, son architecture fondée sur des adaptateurs de flotte et ses mécanismes de coordination du trafic, qui répondent aux contraintes précédentes. Cette architecture présente en outre l’avantage de conserver les systèmes de navigation propres à chaque robot tout en ajoutant une couche commune de gestion et de coordination. Sa mise en œuvre pour les robots MiR250, MiR600 et Omron LD-90 est présentée au chapitre 3.

2.4 Jumeau numérique pour la supervision et la gestion de flottes robotiques

Le jumeau numérique (*Digital Twin*) correspond à une réplique virtuelle d’un système physique dont l’état est continuellement mis à jour à partir des données échangées avec le système réel [7]. Contrairement à une maquette 3D statique, il reflète l’état réel du système et évolue avec lui. Plutôt que d’en exposer une théorie générale, cette section

l’aborde sous l’angle qui structure ce projet : la supervision et la coordination d’une flotte de robots mobiles hétérogènes.

2.4.1 Architecture et exigences d’un jumeau numérique de flotte

Appliqué à une flotte robotique, un jumeau numérique s’organise en trois couches complémentaires : une couche d’acquisition, qui collecte en continu l’état et la position des robots ; une couche de modélisation, qui maintient une représentation virtuelle fidèle de la flotte et de son environnement ; et une couche de supervision, qui restitue ces informations à l’opérateur et, le cas échéant, lui permet d’agir sur le système réel.

Dans le cas d’une flotte *hétérogène*, cette architecture doit satisfaire plusieurs exigences directement issues des contraintes identifiées au chapitre 1 : agréger des données provenant de sources et de protocoles différents (API REST pour les robots MiR, ARCL pour l’Omron LD-90), exprimer les positions des robots dans un référentiel spatial commun, et assurer une mise à jour en temps réel compatible avec la supervision d’opérations réelles. Ces exigences orientent aussi bien le choix de la plateforme de visualisation que celui du mécanisme de synchronisation.

2.4.2 Synchronisation robot réel–modèle virtuel

Le niveau d’intégration entre le système physique et sa représentation numérique se définit par le sens de circulation des données. La littérature distingue trois niveaux, illustrés à la Figure 2.8 :

- **Le modèle numérique (*Digital Model*)** : représentation virtuelle sans échange automatique de données ; toute mise à jour est manuelle.
- **L’ombre numérique (*Digital Shadow*)** : le modèle reçoit automatiquement les données du système physique et reflète son état, mais ne peut pas agir sur lui. Le flux d’information est unidirectionnel.
- **Le jumeau numérique (*Digital Twin*)** : l’échange de données est bidirectionnel et automatique. Le jumeau reflète l’état du système physique et peut également agir sur celui-ci.

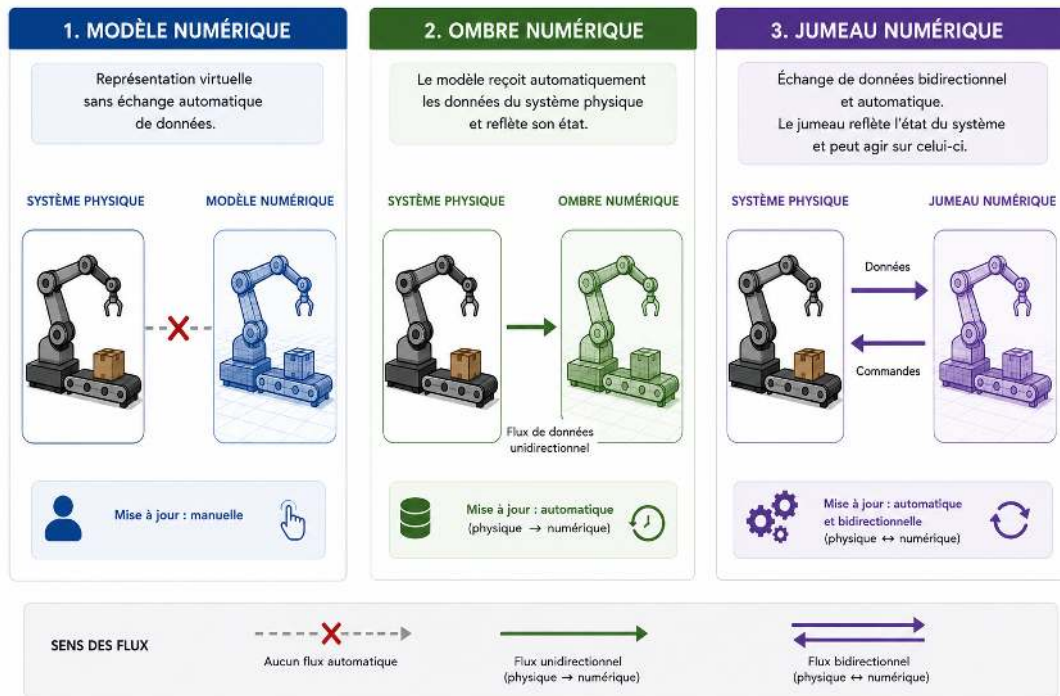


FIGURE 2.8 – Distinction entre modèle, ombre et jumeau numériques [1]

Pour une flotte, la synchronisation repose sur un flux continu d'états et de poses transmis du robot réel vers son modèle. La qualité de la supervision dépend alors de la fréquence de mise à jour et de la latence de ce flux, cette dernière constituant un indicateur de performance retenu pour la validation du projet (Tableau 1.6). Dans le cadre de ce travail, la représentation développée vise à refléter fidèlement l'état de la flotte et à permettre son pilotage, s'inscrivant ainsi dans la logique du jumeau numérique.

2.4.3 Technologies de simulation et de visualisation

Le développement d'un jumeau numérique robotique s'appuie sur deux familles d'outils : les moteurs de simulation, conçus pour reproduire fidèlement la physique et l'interaction des robots avec leur environnement, et les moteurs de jeu, orientés vers la qualité du rendu visuel et la richesse des interfaces utilisateur.

Parmi les moteurs de simulation, Gazebo est largement utilisé dans l'écosystème ROS pour son intégration native et sa gestion réaliste de la physique, tandis que NVIDIA Isaac Sim se distingue par son rendu photoréaliste et ses capacités de simulation avancées exploitant l'accélération GPU. Du côté des moteurs de jeu, Unity et Unreal Engine se caractérisent par la qualité de leur rendu et la souplesse de leurs outils de création d'interfaces, ce qui les rend adaptés aux applications de visualisation et de supervision. Le Tableau 2.8 en résume les principales caractéristiques.

TABLE 2.8 – Principales plateformes de développement de jumeaux numériques

Plateforme	Type	Caractéristique principale
Gazebo	Moteur de simulation	Intégration native avec ROS
NVIDIA Isaac Sim	Moteur de simulation	Rendu photoréaliste, physique accélérée par GPU
Unity	Moteur de jeu	Rendu temps réel et interfaces riches
Unreal Engine	Moteur de jeu	Rendu graphique haute fidélité

Des études comparatives ont analysé ces plateformes pour le développement de jumeaux numériques, en mettant en évidence les compromis entre fidélité physique, réalisme visuel et facilité d'intégration [12]. Le choix dépend ainsi de l'objectif visé : une simulation physique poussée oriente vers un moteur de simulation, tandis qu'un besoin de visualisation et de supervision temps réel favorise un moteur de jeu doté d'un pont de communication vers ROS.

2.4.4 Applications et travaux existants en jumeau numérique de flotte

Le jumeau numérique trouve des applications concrètes dans le domaine des robots mobiles autonomes : il offre un espace virtuel dans lequel les trajectoires, les interactions et les performances peuvent être analysées et optimisées avant ou pendant le fonctionnement réel [7]. Appliqué à une flotte, il constitue un outil de supervision globale, capable de visualiser en temps réel l'état de l'ensemble des robots et d'assister l'opérateur dans la prise de décision.

Toutefois, une part importante des travaux existants se limite soit à un seul type de robot, soit à un usage de simulation hors ligne, et aborde moins la supervision temps réel d'une flotte *hétérogène* en exploitation réelle. Ce constat définit l'espace dans lequel se positionne le présent projet.

2.4.5 Positionnement de l'approche proposée

Au regard des exigences précédentes, l'approche retenue consiste à développer le jumeau numérique sous Unity, utilisé comme couche de supervision et de visualisation temps réel de la flotte. Ce choix répond à l'exigence de rendu interactif (Section 2.4.3) et présente l'avantage d'un pont de communication direct avec ROS 2 via le *ROS-TCP-Connector*, qui assure la synchronisation des états et des positions des robots (Section 2.4.2). Le jumeau numérique se place ainsi au-dessus de la couche d'orchestration Open-RMF : cette dernière coordonne la flotte, tandis qu'Unity en fournit la représentation supervisable. La conception et l'implémentation de ce jumeau numérique font l'objet du chapitre 4.

2.5 Synthèse et positionnement du projet

L'étude de l'état de l'art a permis de dégager les principaux constats sur lesquels repose ce projet.

Premièrement, les robots mobiles autonomes constituent aujourd'hui une brique essentielle de l'Industrie 4.0, mais leur valeur ne se révèle pleinement que lorsqu'ils sont

exploités collectivement, au sein d’une flotte coordonnée. Or, la coordination d’une flotte soulève des défis d’allocation de tâches, de gestion du trafic et, surtout, d’interopérabilité entre des robots de constructeurs différents.

Deuxièmement, les travaux existants abordent ces défis de manière partielle. Certains se concentrent sur l’allocation de tâches à l’aide de méthodes avancées telles que l’apprentissage par renforcement [9], sans nécessairement traiter l’hétérogénéité matérielle des flottes. D’autres proposent des frameworks de planification centralisée [11] ou des déploiements de flottes homogènes en conditions réelles [8]. Les efforts de standardisation, comme la norme VDA5050 [10], ouvrent la voie à l’interopérabilité, mais leur adoption reste limitée aux robots qui l’implémentent nativement, ce qui exclut de nombreuses plateformes déjà déployées.

Troisièmement, le jumeau numérique est mobilisé pour la supervision et l’analyse des systèmes robotiques [7], mais il est rarement couplé à un système de gestion de flotte réel et hétérogène. La plupart des travaux traitent la simulation ou la gestion de flotte de façon indépendante.

C’est à l’intersection de ces trois constats que se positionne ce projet. Il vise à concevoir un système de gestion de flotte réellement interopérable, capable d’intégrer des robots de fabricants différents (MiR et Omron) sans modifier leurs systèmes embarqués, en s’appuyant sur le framework ouvert Open-RMF déployé sur ROS 2. Ce système est en outre couplé à un jumeau numérique développé sous Unity, assurant la visualisation et la supervision en temps réel de l’ensemble de la flotte. La contribution de ce travail réside donc dans l’association de deux volets souvent traités séparément dans la littérature : l’interopérabilité multi-fabricant d’une part, et l’intégration d’un jumeau numérique de supervision d’autre part.

Conclusion

Ce chapitre a établi le socle théorique et technologique du projet. Après avoir présenté le contexte de l’Industrie 4.0 et de la *Smart Factory*, nous avons étudié les robots mobiles autonomes, leurs principes de navigation et leur exploitation en flotte. Nous avons ensuite analysé la gestion de flotte robotique sous l’angle de l’allocation de tâches, de l’interopérabilité et des frameworks existants, avant d’aborder le jumeau numérique et ses différentes déclinaisons. La synthèse de cet état de l’art a mis en évidence un besoin peu couvert par la littérature : celui d’une solution alliant gestion de flotte interopérable et supervision par jumeau numérique. Le chapitre suivant est consacré à l’analyse des besoins et à la conception de l’architecture répondant à cette problématique.

Chapitre 3 : Conception et implémentation du système

Chapitre 3 : Conception et implémentation du système

Introduction

Après avoir défini le contexte du projet et étudié l'état de l'art des technologies concernées, ce chapitre est consacré à la conception et à l'implémentation de la solution proposée. Nous commençons par formaliser les besoins auxquels le système doit répondre, en distinguant les besoins fonctionnels des besoins non fonctionnels. Nous présentons ensuite l'architecture globale retenue et justifions les principaux choix technologiques au regard des exigences identifiées. Nous détaillons enfin l'implémentation de la solution, en abordant successivement la couche d'intégration des robots, l'orchestration de la flotte à l'aide d'Open-RMF, la mise en place d'un référentiel spatial commun, ainsi que le développement du jumeau numérique sous Unity. La validation expérimentale de la solution, à travers différents scénarios de mission, fait l'objet du chapitre suivant. .

3.1 Analyse des besoins

Avant de concevoir la solution, il est essentiel d'identifier précisément les besoins auxquels le système doit répondre. Ces besoins se répartissent en deux catégories : les besoins fonctionnels, qui décrivent les services que le système doit rendre, et les besoins non fonctionnels, qui définissent les contraintes de qualité et de performance.

3.1.1 Besoins fonctionnels

Le système à concevoir doit permettre la gestion et la supervision d'une flotte hétérogène de robots mobiles autonomes. Les principales fonctionnalités attendues sont les suivantes :

- **Intégration des robots** : communiquer avec des robots de constructeurs différents (MiR et Omron) à travers leurs protocoles natifs, sans modifier leurs systèmes embarqués.
- **Gestion des missions** : envoyer des missions aux robots, suivre leur exécution et récupérer leur état en temps réel.
- **Coordination de la flotte** : orchestrer les déplacements des robots, planifier le trafic et éviter les conflits de trajectoire lorsqu'ils partagent le même espace.
- **Supervision en temps réel** : visualiser la position, l'état, le niveau de batterie et les missions en cours de chaque robot.
- **Représentation numérique** : disposer d'un jumeau numérique reproduisant fidèlement l'environnement et l'état de la flotte en temps réel.
- **Contrôle depuis l'interface** : permettre à l'opérateur de commander les robots directement depuis l'interface de supervision.

3.1.2 Besoins non fonctionnels

Au-delà des services rendus, le système doit satisfaire un ensemble d'exigences de qualité :

- **Interopérabilité** : intégrer des robots hétérogènes au sein d'une architecture commune, indépendamment de leur fabricant.
- **Modularité et extensibilité** : permettre l'ajout de nouveaux robots ou de nouvelles fonctionnalités avec un effort d'intégration minimal.
- **Temps réel** : assurer une communication et une synchronisation avec une latence suffisamment faible pour refléter fidèlement l'état du système physique.
- **Fiabilité** : garantir un fonctionnement stable et une communication robuste entre les différents composants.
- **Ergonomie** : offrir une interface claire et intuitive, facilitant la supervision et le contrôle par l'opérateur.
- **Portabilité** : pouvoir déployer la solution sur les machines de l'environnement de travail sans dépendance matérielle contraignante.

3.2 Architecture globale de la solution

L'architecture globale de la solution a été conçue de manière à répondre aux besoins identifiés, en particulier l'interopérabilité entre des robots hétérogènes et la supervision par un jumeau numérique. Elle s'organise autour d'une couche centrale d'orchestration, reposant sur le framework Open-RMF déployé sur ROS 2, qui joue le rôle de chef d'orchestre de l'ensemble du système.

Le système peut être décomposé en trois niveaux complémentaires :

- **Le niveau physique** : il regroupe les robots réels de la flotte (MiR250, MiR600 et Omron LD-90), chacun disposant de son propre système de navigation et de son interface de communication native.
- **Le niveau intergiciel** : il assure la liaison entre les robots et la couche d'orchestration. Des adaptateurs (drivers) traduisent les commandes issues d'Open-RMF vers les protocoles propres à chaque robot (API REST pour les MiR, ARCL pour l'Omron), et publient les informations d'état sous forme de *topics* ROS 2.
- **Le niveau supervision** : il correspond au jumeau numérique développé sous Unity, qui reçoit les données de la flotte via un pont de communication et offre à l'opérateur une visualisation et un contrôle en temps réel.

La Figure 3.1 présente cette architecture et les échanges entre ses différents niveaux. Cette organisation en couches garantit la modularité du système : chaque robot est intégré indépendamment des autres, et l'ajout d'une nouvelle plateforme se limite au développement de son adaptateur, sans impact sur le reste de l'architecture.

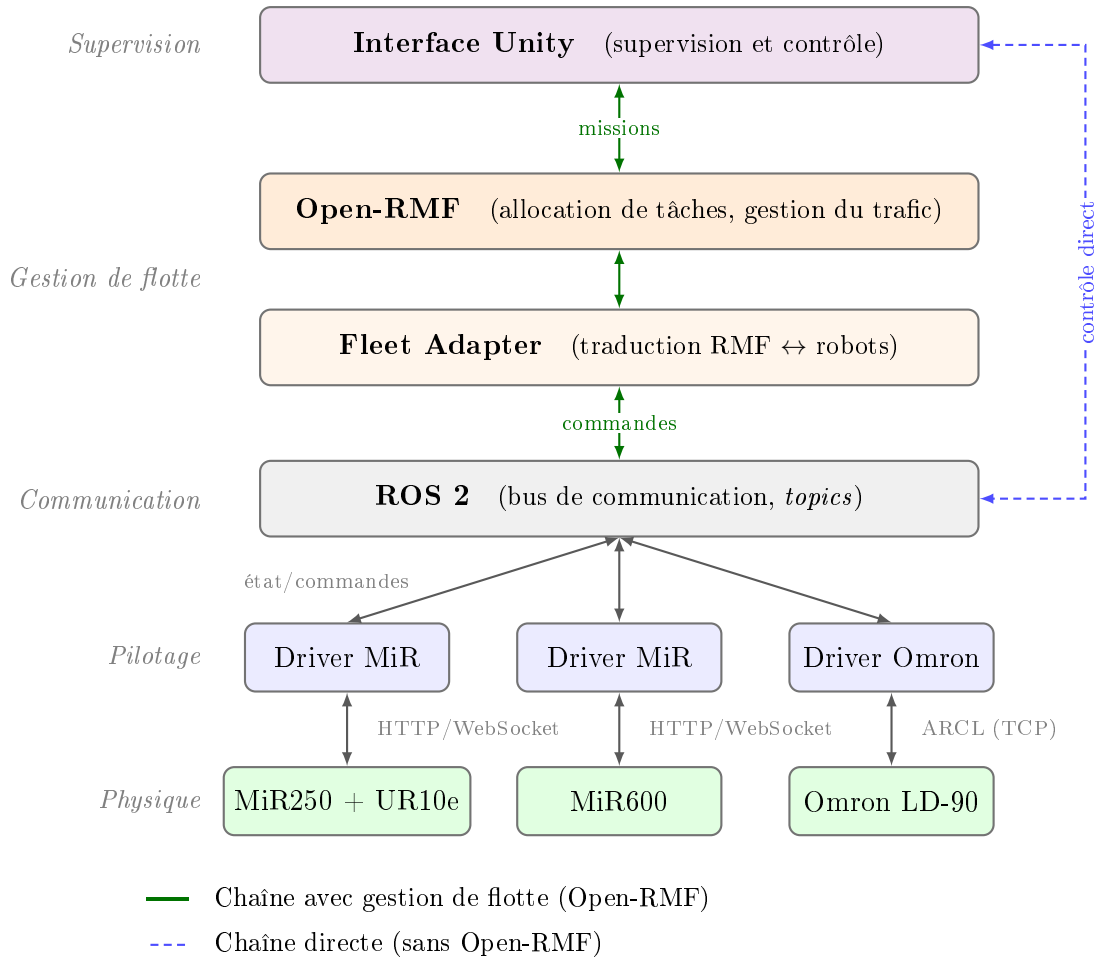


FIGURE 3.1 – Architecture globale de la solution et ses deux chaînes de contrôle

3.3 Couche d'intégration des robots

La coordination d'une flotte hétérogène suppose d'abord de rendre chaque robot observable et pilotable de manière uniforme, indépendamment de son constructeur et sans modifier son système embarqué. Cette couche d'intégration repose sur **ROS 2 (Jazzy)**, retenu comme socle de communication commun : chaque robot est encapsulé par un driver qui traduit son protocole propriétaire en un ensemble de topics ROS 2 normalisés (état, position, capteurs) et convertit les commandes ROS 2 vers son interface native. Les robots MiR et l'Omron LD-90 exposant des interfaces différentes, deux drivers distincts ont été mis en œuvre.

3.3.1 Driver des robots MiR

Les robots MiR (MiR250 et MiR600) exposent deux interfaces complémentaires, exploitées par le driver à travers **deux canaux indépendants**, chacun porté par un nœud ROS 2 dédié (Figure 3.2) :

- **Un canal WebSocket continu** (via *rosbridge*, sur `ws://<host>:9090`) pour les données temps réel et les commandes de mouvement.
- **Un canal REST à la demande** (API MiR `/api/v2.0.0`, port 80, en HTTP) pour la gestion des missions.

Ce découpage sépare le flux temps réel (continu, haute fréquence) de la gestion des missions (ponctuelle, en requête/réponse). Il permet d'intégrer indifféremment le MiR250 et le MiR600, qui partagent la même architecture logicielle et ne se distinguent que par leur adresse réseau.

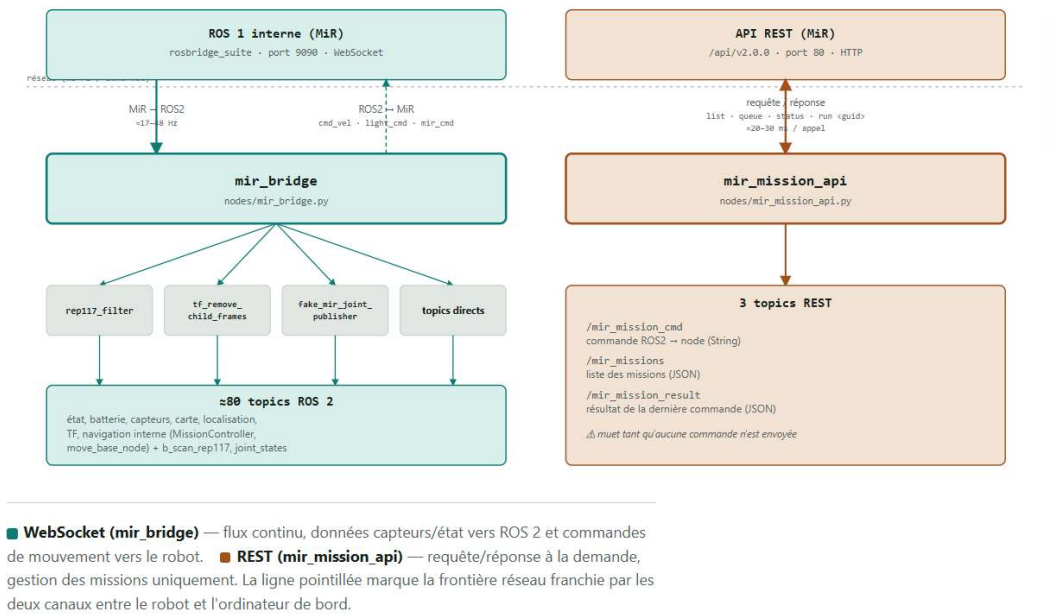


FIGURE 3.2 – Architecture du driver MiR : canal WebSocket (temps réel) et canal REST (missions)

Le nœud mir_bridge (canal temps réel)

Le nœud mir_bridge constitue le cœur du driver. Il établit la connexion WebSocket avec le rosbridge embarqué du robot, puis republie les topics internes du MiR sous forme de topics ROS 2 natifs, tout en retransmettant vers le robot les commandes issues de ROS 2. La communication est **bidirectionnelle** : dans le sens robot → ROS 2, il expose l'état, la batterie, les capteurs, la carte, la localisation, les transformations TF et les données de navigation ; dans le sens ROS 2 → robot, il transmet les commandes de vitesse et de pose. Le Tableau 3.1 en résume les principales interfaces.

TABLE 3.1 – Principales interfaces ROS 2 du nœud mir_bridge

Sens	Topic (exemples)	Contenu
Robot → ROS 2	/robot_status, /odom, /map	État, batterie, odométrie, carte
Robot → ROS 2	/tf, /tf_static	Transformations entre repères
Robot → ROS 2	/f_scan, /b_scan	Données des télémètres laser
ROS 2 → Robot	/cmd_vel	Commandes de vitesse

Pour assurer la compatibilité entre l'environnement du robot et ROS 2, le nœud mir_bridge réalise automatiquement quelques adaptations. Il ajoute à chaque robot un

préfixe sur ses repères TF (par exemple `mir250/base_link`) pour éviter les conflits de noms lorsque plusieurs robots coexistent, redatate les messages dont l'horodatage est nul afin qu'ils soient acceptés par ROS 2, et applique la bonne politique de qualité de service (QoS) aux topics latched comme `/map` et `/tf_static`, sans laquelle ces données ne seraient pas reçues. Enfin, il convertit les commandes de vitesse au format attendu par le robot avant de les lui transmettre. Afin de limiter la charge réseau, le nœud n'active la souscription à un flux du robot que lorsqu'au moins un abonné ROS 2 est présent, à l'exception des topics latched qui sont souscrits immédiatement.

Le nœud `mir_mission_api` (canal REST)

Le nœud `mir_mission_api` gère les missions du robot en interrogeant l'API REST du MiR (`/api/v2.0.0`). Il reçoit des commandes textuelles sur un topic dédié et effectue les appels HTTP correspondants, puis republie les résultats sur ROS 2. Les commandes prises en charge sont résumées dans le Tableau 3.2.

TABLE 3.2 – Commandes gérées par le nœud `mir_mission_api`

Commande	Action
<code>list</code>	Récupère et publie la liste des missions disponibles
<code>run <guid></code>	Ajoute la mission indiquée à la file d'exécution du robot
<code>queue</code>	Publie le contenu de la file de missions
<code>status</code>	Publie l'état courant du robot

Ce nœud communique avec le robot sur un canal distinct du flux temps réel : les missions sont gérées ponctuellement, en requête/réponse, sans interférer avec la boucle de contrôle continue assurée par `mir_bridge`.

Nœuds de post-traitement :

Autour de ces deux nœuds principaux, plusieurs nœuds complètent le driver afin de rendre les données directement exploitables par la suite de la chaîne (navigation, localisation, visualisation) :

- `rep117_filter` : met les scans laser en conformité avec la norme REP-117, en codant correctement les mesures hors portée (trop proche, trop loin) attendues par les outils de navigation.
- `tf_remove_child_frames` : filtre certaines transformations de l'arbre TF issu du robot, afin d'éviter les conflits avec les repères introduits par la couche de localisation commune de la flotte.
- Publieur d'états articulaires : génère les *joint_states* des roues et des roulettes, nécessaires à l'affichage du modèle du robot dans les outils de visualisation.

L'ensemble de ces nœuds expose chaque robot MiR à travers une interface ROS 2 homogène, prête à être exploitée par la couche d'orchestration.

3.3.2 Driver de l'Omron LD-90

Contrairement aux robots MiR intégrés dans le projet, l'Omron LD-90 ne dispose pas d'une interface ROS 2 utilisée directement par l'architecture proposée. Sa communication avec les systèmes externes repose sur le protocole propriétaire **ARCL** (Advanced Robotics Command Language), qui permet l'échange de commandes et d'informations d'état avec le robot au moyen d'une connexion TCP.

Afin d'intégrer l'Omron LD-90 dans l'écosystème ROS 2 et, par la suite, dans l'architecture de gestion de flotte basée sur Open-RMF, un driver spécifique nommé `omron_ld_driver` a été développé. Ce driver constitue une couche d'adaptation entre les interfaces ROS 2 et le protocole ARCL. Il assure une communication bidirectionnelle : les commandes reçues depuis ROS 2 sont converties en commandes ARCL avant d'être transmises au robot, tandis que les informations retournées par celui-ci sont interprétées puis publiées sous forme d'interfaces ROS 2.

Le driver permet notamment de récupérer les informations relatives à la localisation, à la batterie et à l'état du robot, ainsi que de transmettre des commandes de navigation et de contrôle. Cette organisation permet d'isoler les spécificités du protocole propriétaire du reste de l'architecture logicielle selon la chaîne suivante :

ROS 2 ↔ `omron_ld_driver` ↔ ARCL/TCP ↔ Omron LD-90

Dans l'architecture développée au CNIMI, le driver est exécuté sous le namespace `/omron`. Il constitue ainsi l'interface logicielle entre le robot physique et les composants ROS 2 chargés de son intégration et de son orchestration.

Architecture en couches

Le driver a été organisé selon trois couches logiques afin de séparer les responsabilités liées à la communication réseau, à l'interprétation du protocole ARCL et aux interfaces ROS 2. Cette organisation facilite la maintenance du logiciel et permet de faire évoluer indépendamment les différentes composantes du pilote. Le Tableau 3.3 détaille le rôle de chaque couche, et la Figure 3.3 en illustre l'agencement et le flux de communication bidirectionnel.

TABLE 3.3 – Architecture logicielle du driver Omron LD-90

Couche	Composant	Responsabilité
Transport	<code>arcl_client</code>	Établissement de la connexion TCP, authentification, envoi et réception des messages et gestion de la reconnexion
Traduction	<code>arcl_status_parser</code>	Interprétation des messages ARCL et conversion des informations reçues vers une représentation structurée
ROS 2	<code>omron_ld_driver_node</code>	Gestion des paramètres et des interfaces ROS 2, ainsi que traduction des commandes ROS 2 vers ARCL

Les deux premières couches, `arcl_client` et `arcl_status_parser`, sont regroupées dans une bibliothèque `arcl_core` indépendante de ROS 2. Le nœud `omron_ld_driver_node`,

implémenté par la classe `OmronLdDriver`, utilise cette bibliothèque et assure l'intégration avec les interfaces ROS 2. Ce découpage isole le transport des données, leur interprétation et leur exposition au sein de ROS 2, de sorte que le protocole constructeur reste confiné aux couches basses.

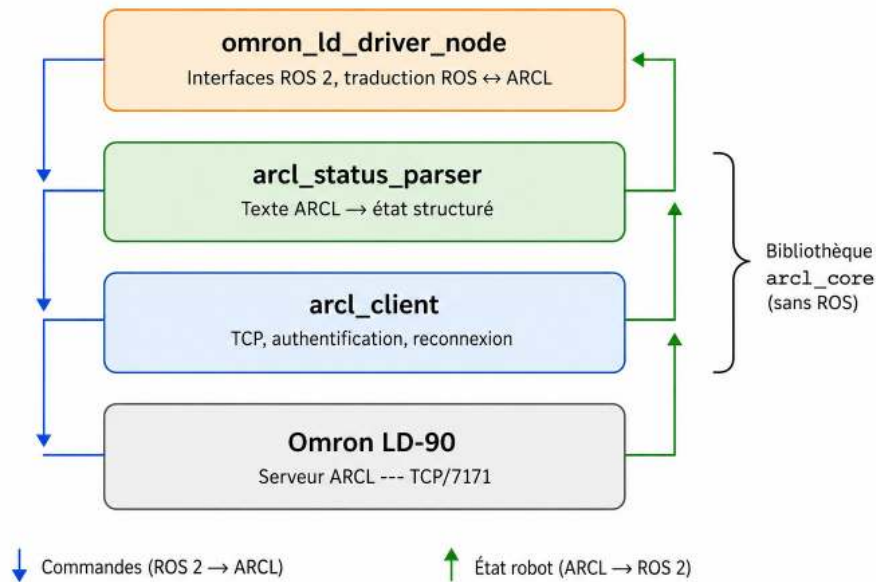


FIGURE 3.3 – Architecture en couches du driver Omron LD-90 et flux bidirectionnel

Comme le montre la Figure 3.3, le flux de communication est bidirectionnel. Dans le sens descendant (commandes), les ordres issus des interfaces ROS 2 sont traités par le nœud, convertis au format ARCL, puis transmis au robot via la connexion TCP. Dans le sens montant (état), les informations retournées par le robot sont reçues par le client ARCL, interprétées par le parseur, puis publiées sous forme de topics ROS 2.

Fonctionnement du driver

Le fonctionnement du pilote repose sur une chaîne de traitement bidirectionnelle. Lors du démarrage, le nœud ROS 2 initialise ses paramètres et tente d'établir une connexion avec le serveur ARCL du robot. Une fois la connexion établie, un mécanisme de réception permet de récupérer continuellement les messages transmis par le robot.

Les messages reçus sont ensuite transmis au parseur `arcl_status_parser`, qui extrait les informations utiles et les convertit vers une représentation structurée de l'état du robot. Ces données sont ensuite utilisées par le nœud ROS 2 afin d'alimenter les interfaces de publication correspondantes.

Dans le sens inverse, les commandes reçues sur les interfaces ROS 2 sont converties en commandes ARCL avant d'être transmises au robot. Cette séparation entre communication, interprétation et interfaces ROS 2 permet de conserver une architecture modulaire et de limiter l'impact des spécificités du protocole propriétaire sur le reste du système.

Le driver intègre également un mécanisme de reconnexion permettant de rétablir la communication en cas de perte de la connexion TCP. Cette fonctionnalité est importante pour assurer la robustesse de la communication entre le système de contrôle et le robot.

Dans l'implémentation développée, les commandes de navigation sont traduites vers les commandes ARCL correspondantes. Par exemple, une commande de déplacement vers

la position (5 m, 2 m) peut être convertie en une commande de type gotoPoint 5000 2000, conformément au format utilisé dans l'interface ARCL mise en œuvre par le driver.

Une particularité de l'intégration concerne le contrôle par vitesse. Dans l'implémentation retenue, le contrôle continu de type cmd_vel n'est pas utilisé comme interface native de commande du robot. Les commandes de vitesse reçues par le driver sont donc converties en déplacements discrets. À partir de la dernière position connue du robot, une nouvelle consigne de déplacement est calculée puis transmise au moyen d'une commande gotoPoint.

Cette stratégie permet de conserver une interface ROS 2 compatible avec les besoins de l'architecture globale tout en tenant compte des possibilités offertes par l'interface ARCL utilisée pour communiquer avec le LD-90.

Normalisation des données

L'une des fonctions importantes du driver consiste à transformer les données issues du protocole propriétaire vers des représentations exploitables dans ROS 2. Les informations reçues du robot sont ainsi interprétées puis normalisées avant leur publication.

Les grandeurs utilisées par les composants ROS 2 sont exprimées dans les unités conventionnelles du système, notamment en mètres pour les positions et en radians pour les orientations. Les valeurs brutes retournées par l'interface ARCL peuvent également être conservées pour les besoins du diagnostic et de la vérification de la communication.

Cette normalisation permet de découpler les unités et conventions propres au protocole ARCL des composants situés en aval du driver. Elle facilite notamment l'intégration ultérieure du robot dans le référentiel spatial commun utilisé par Open-RMF.

Interfaces ROS 2

Le driver expose les principales informations du robot et les commandes nécessaires à son intégration dans l'architecture ROS 2 au moyen de topics, de services et d'une action. Le Tableau 3.4 présente les principales interfaces utilisées dans le système développé.

TABLE 3.4 – Principales interfaces ROS 2 du driver Omron LD-90 (namespace /omron)

Type	Interface	Fonction
Topic publié	/omron/odom	Publication de la pose issue de la localisation du robot
Topic publié	/omron/battery_state	Publication de l'état de la batterie
Topic publié	/omron/localization_score	Publication du score de confiance associé à la localisation
Topic souscrit	/omron/cmd_vel	Réception des commandes de déplacement converties en consignes discrètes
Topic souscrit	/omron/goto_point	Navigation vers une position cible
Service	/omron/stop	Arrêt du robot
Service	/omron/dock	Commande de mise en charge
Service	/omron/undock	Commande de sortie de la station de charge
Action	/omron/go_to_goal	Navigation vers un objectif avec suivi de l'exécution

La pose publiée sur /omron/odom est issue des informations de localisation fournies par le robot via son interface de communication. Cette distinction est importante pour la suite du projet, notamment lors de l'établissement du référentiel spatial commun entre les différentes plateformes de la flotte.

Intégration du driver dans l'architecture de la flotte

Une fois le driver opérationnel, l'Omron LD-90 peut être considéré par les composants ROS 2 comme une plateforme intégrée à l'architecture logicielle commune. Le driver joue alors le rôle d'interface entre les services propriétaires du robot et les composants chargés de l'orchestration de la flotte.

Cette intégration présente un intérêt particulier dans le contexte d'une flotte hétérogène. Les robots MiR communiquent au moyen de leur API REST et d'un canal Web-Socket, tandis que l'Omron LD-90 repose sur le protocole ARCL/TCP. Le driver permet de masquer ces différences au niveau des couches supérieures, en fournissant des interfaces ROS 2 conformes à l'architecture commune. La couche d'intégration peut ainsi être représentée de manière simplifiée par :

$$\text{Open-RMF} \leftrightarrow \text{ROS 2} \leftrightarrow \text{omron_ld_driver} \leftrightarrow \text{ARCL/TCP} \leftrightarrow \text{Omron LD-90}$$

En définitive, le développement de ce driver a permis d'établir une communication bidirectionnelle complète entre l'Omron LD-90 et ROS 2 : établissement de la connexion ARCL/TCP, interprétation et publication des informations d'état sous forme d'interfaces ROS 2, et conversion des commandes de navigation vers le protocole du robot. L'Omron LD-90 est ainsi intégré à l'infrastructure ROS 2 commune sans modification de son système embarqué, au même titre que les robots MiR. La couche d'orchestration présentée dans la section suivante peut dès lors traiter les trois robots de manière uniforme.

3.4 Orchestration de la flotte avec Open-RMF

Une fois les trois robots exposés à travers une interface ROS 2 homogène (Section 3.3), il reste à les coordonner : allouer les missions, planifier les déplacements et gérer le trafic lorsque les robots partagent le même espace. Ces besoins de coordination et d'interopérabilité (Section 3.1) appellent une couche d'orchestration indépendante des constructeurs. Conformément à l'analyse comparative menée dans l'état de l'art (Chapitre 2), **Open-RMF** a été retenu à cet effet : bâti sur ROS 2, il orchestre les missions et coordonne le trafic de flottes hétérogènes au moyen d'adaptateurs de flotte (*fleet adapters*), tout en préservant les systèmes de navigation propriétaires de chaque robot.

L'environnement a été déployé sous **ROS 2 Jazzy** à partir du méta-paquet `ros-jazzy-rmf-dev`, qui fournit les bibliothèques RMF et les modules Python nécessaires au développement du Fleet Adapter. L'architecture retenue distingue **deux flottes** : la flotte `mir_fleet`, regroupant les robots MiR250 et MiR600 (interface REST), et la flotte `omron_fleet`, dédiée à l'Omron LD-90 (interface ROS 2 via son driver). Ce découpage reflète l'hétérogénéité protocolaire de la flotte ; il permet en outre au Dispatcher d'Open-RMF de mettre les deux flottes en concurrence lors de l'allocation d'une mission. La Figure 3.4 situe ces composants dans la chaîne de contrôle, du client de missions jusqu'à l'exécution physique par les robots.

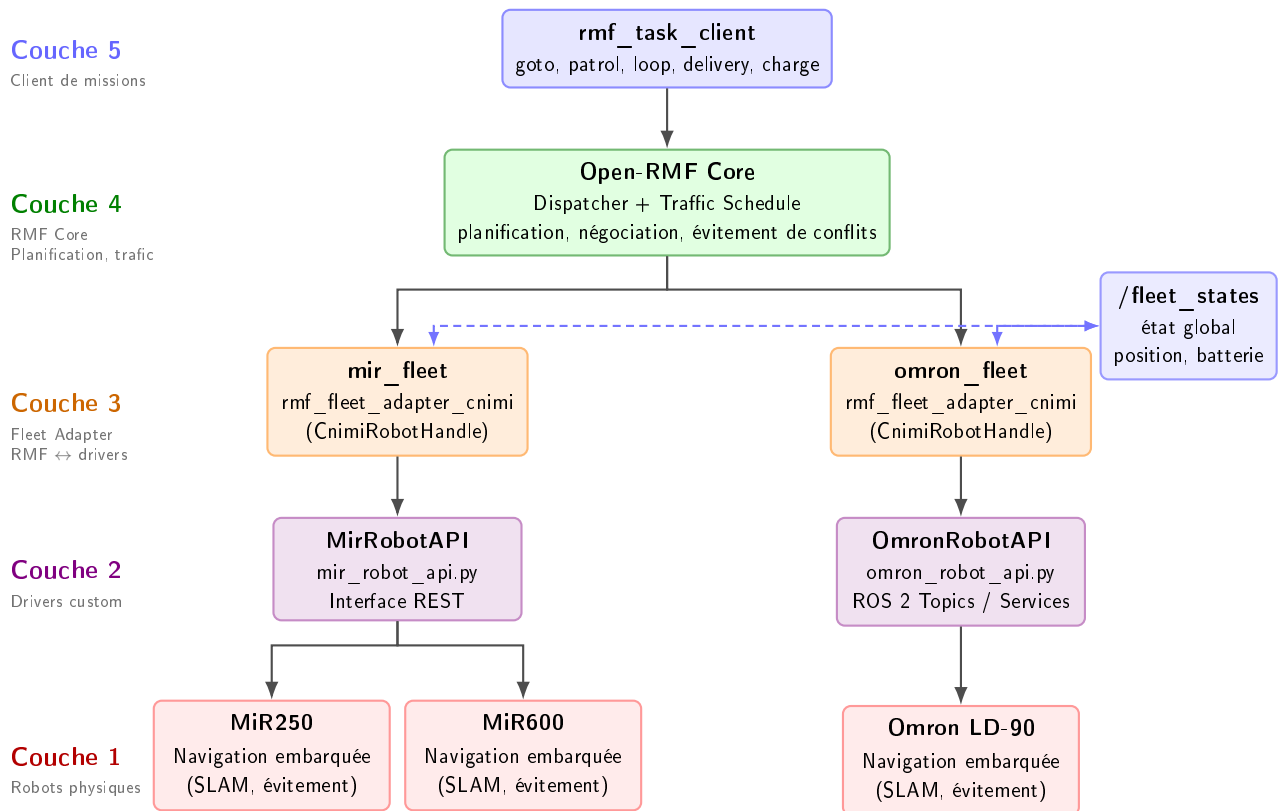


FIGURE 3.4 – Architecture d'intégration d'Open-RMF (deux flottes) et chaîne d'exécution d'une mission

3.4.1 Carte et graphe de navigation

La coordination du trafic par Open-RMF repose sur un graphe de navigation décrivant les déplacements autorisés dans l'environnement partagé. La carte du laboratoire CNIMI

a été récupérée depuis le robot MiR250 via son API REST, puis importée dans l'outil *Traffic Editor* (Figure 3.5). Les voies de circulation (*lanes*), les stations de charge et les principaux points de passage y ont été annotés, avant la génération du graphe de navigation à l'aide de *building_map_generator*. Le Tableau 3.5 présente les principaux points d'intérêt du graphe.

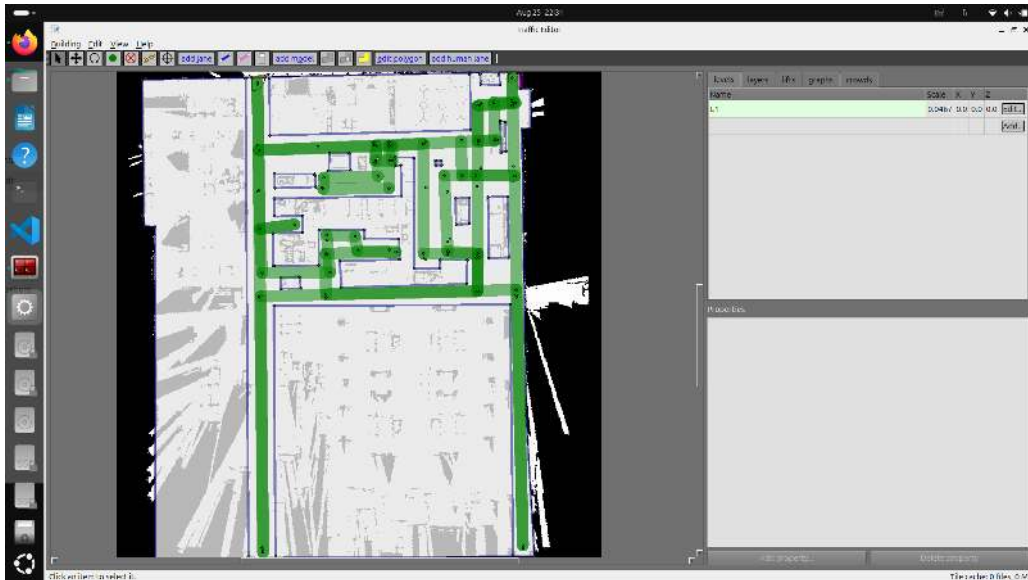


FIGURE 3.5 – Annotation de la carte CNIMI et des voies de circulation dans Traffic Editor

TABLE 3.5 – Points d'intérêt du graphe de navigation, par catégorie

Waypoint(s)	Fonction
mir_charging_station	Station de charge des robots MiR
omron_charging_station	Station de charge de l'Omron LD-90
Parking_mir	Zone de stationnement
door_1	Point de passage (porte)
L10-L13, L20-L24	Points de navigation intermédiaires (voies de circulation)
L50-L53	Points de navigation (zone secondaire)

3.4.2 Le Fleet Adapter

Afin de permettre à Open-RMF de communiquer avec les robots physiques, un package ROS 2 nommé *rmf_fleet_adapter_cnimi*, développé sous forme de package *ament_python*, a été mis en place. Son rôle est d'assurer l'adaptation entre les interfaces d'Open-RMF et les interfaces de communication propres aux différentes plateformes de la flotte. Les robots intégrés au projet ne communiquant pas nativement avec Open-RMF, le Fleet Adapter constitue une couche intermédiaire permettant de traduire les commandes et les informations échangées entre les deux environnements. Chaque flotte est configurée à partir d'un fichier *fleet_config.yaml*, qui regroupe notamment les paramètres nécessaires à l'intégration des robots, à leur navigation et à leur gestion au sein de RMF. La structure

générale de cette configuration est présentée dans le Tableau 3.6. Le package est organisé autour de plusieurs modules complémentaires :

- **fleet_adapter.py** : constitue le point d'entrée de l'adaptateur ROS 2. Il initialise les composants nécessaires à l'intégration de la flotte dans RMF, charge les paramètres et le graphe de navigation associés à la flotte, crée les interfaces de communication avec les robots, puis assure leur enregistrement auprès d'Open-RMF. Il participe également à la remontée périodique des informations d'état, notamment la position et le niveau de batterie.
- **mir_robot_api.py** : assure l'adaptation des commandes destinées aux robots MiR. Les commandes issues de la couche RMF sont traduites en appels vers l'API REST v2.0.0 des robots MiR. Cette interface est utilisée pour l'intégration du MiR250 et du MiR600 et permet notamment de transmettre des missions, d'arrêter un robot et de récupérer ses informations d'état.
- **omron_robot_api.py** : assure l'intégration de l'Omron LD-90. Contrairement à l'interface utilisée pour les robots MiR, cette couche s'appuie sur le driver ROS 2 développé précédemment. Elle récupère les informations publiées par le driver, notamment la position, la batterie et l'état du robot, et transmet les commandes de navigation vers les interfaces ROS 2 correspondantes.
- **battery_monitor.py** : constitue un module dédié à la surveillance du niveau de batterie des robots. Lorsque le niveau de charge d'un robot atteint le seuil défini dans la configuration, le module peut déclencher la procédure de retour vers la station de recharge correspondante.

Cette organisation permet de séparer les responsabilités entre la gestion de flotte, l'adaptation aux interfaces propriétaires et la surveillance des ressources. Elle facilite également l'extension de la solution : l'intégration d'une nouvelle plateforme peut être réalisée en ajoutant une interface spécifique au robot tout en conservant la logique générale du Fleet Adapter.

L'architecture obtenue peut ainsi être résumée par les deux chaînes de communication suivantes :

Open-RMF $\leftrightarrow$ rmf_fleet_adapter_cnimi $\leftrightarrow$ mir_robot_api.py $\rightarrow$ API REST $\rightarrow$ MiR250 / MiR600

Open-RMF $\leftrightarrow$ rmf_fleet_adapter_cnimi $\leftrightarrow$ omron_robot_api.py $\rightarrow$ driver ROS 2 $\rightarrow$ ARCL/TCP $\rightarrow$ Omron LD-90

Le Fleet Adapter constitue ainsi la couche d'intégration entre l'orchestrateur Open-RMF et les systèmes de commande propres aux robots. Il permet à RMF de manipuler les différentes plateformes à travers une architecture commune, tout en conservant leurs interfaces et leurs systèmes embarqués d'origine.

TABLE 3.6 – Configuration des robots des deux flottes

Flotte	Robot	Interface	Rayon
cnimi_fleet_mir	MiR250	REST	0,50 m
cnimi_fleet_mir	MiR600	REST	0,70 m
cnimi_fleet_omron	Omron LD-90	ROS 2	0,35 m

Open-RMF ne commande aucun robot directement : il délègue chaque ordre à un objet *RobotCommandHandle* fourni par l'intégrateur, ici la classe `CnimiRobotHandle`. Une

instance est créée par robot enregistré, et RMF y appelle trois méthodes principales (Tableau 3.7). Le développement s’est appuyé sur l’API bas niveau `Adapter.add_fleet()`, la classe `CnimiRobotHandle` assurant la réception des missions, la transmission des commandes de navigation, la gestion des arrêts et le retour à la station de charge, ainsi que la mise à jour continue de la position et de la batterie des robots dans RMF.

TABLE 3.7 – Méthodes du *RobotCommandHandle* et leur traduction

Méthode RMF	Rôle	Traduction CNIMI
<code>follow_new_path()</code>	RMF transmet une liste de waypoints à atteindre	Extraction du waypoint cible, transformation de repère, puis appel de <code>api.navigate(x, y, theta)</code>
<code>stop()</code>	Arrêt immédiat (négociation de trafic)	Appel de <code>api.stop()</code> : pause REST pour les MiR, service ROS pour l’Omron
<code>dock()</code>	Docking / recharge	Confirmation immédiate (recharge gérée par le module de surveillance batterie)

Au lancement, pour chaque robot déclaré, une instance d’API est créée puis testée : un robot injoignable est ignoré sans interrompre le nœud, de sorte que la flotte reste fonctionnelle même en l’absence d’un robot. La position initiale de chaque robot est lue puis convertie dans le repère RMF, et le robot est enregistré auprès de son *fleet handle*. La publication du topic `/fleet_states` valide la communication entre le Fleet Adapter et Open-RMF ; ce topic constitue également le point d’intégration utilisé pour la visualisation de la flotte dans le jumeau numérique (Section 3.5).

3.4.3 Cycle de vie d’une navigation

L’exécution d’une commande de déplacement, du planificateur RMF jusqu’au mouvement physique du robot, suit la séquence suivante :

1. RMF planifie un chemin et appelle `follow_new_path()` sur le `CnimiRobotHandle` du robot sélectionné.
2. Le handle extrait le waypoint cible, le convertit du repère RMF vers le repère du robot, puis appelle `api.navigate()`.
3. Côté MiR, l’API crée une position et une mission temporaires puis les place dans la file de missions du robot ; côté Omron, la destination est publiée directement sur le topic de navigation du driver.
4. Un fil de surveillance interroge périodiquement l’état d’avancement ; dès que le robot signale la fin du déplacement, RMF en est informé.
5. En parallèle, une boucle de mise à jour (à 1 Hz) relit la position et le niveau de batterie du robot, les convertit dans le repère RMF et les transmet à Open-RMF.

3.4.4 Calibration des repères robot–RMF

Chaque robot navigue dans son propre repère de carte, calibré indépendamment sur le logiciel du fabricant, alors qu’Open-RMF raisonne dans un repère global unique partagé

par toute la flotte. Une phase de calibration a donc été menée pour assurer la correspondance entre les coordonnées propres à chaque robot et celles du graphe RMF, répondant ainsi à la contrainte spatiale identifiée au chapitre 1. Pour chaque robot, un décalage (dx, dy) a été mesuré en le plaçant sur un point physique connu (Tableau 3.8).

TABLE 3.8 – Décalages de calibration entre le repère de chaque robot et le repère RMF

Robot	dx (m)	dy (m)
MiR250	-11,970	-69,150
MiR600	-8,300	-41,450
Omron LD-90	28,133	-20,711

Deux fonctions réciproques assurent la conversion : `robot_to_rmf()` exprime la position du robot dans le repère RMF (pour la remontée d'état), et `rmf_to_robot()` convertit une destination du repère RMF vers le repère du robot (pour l'envoi des commandes de navigation).

3.4.5 Le client de missions

Un package `rmf_task_client` a été développé pour créer et envoyer des missions vers Open-RMF, sans communiquer directement avec les robots. Il regroupe les modules dédiés à la définition des missions, à la génération des requêtes RMF et à la communication avec le Dispatcher. Les types de missions pris en charge sont : *goto* (déplacement vers un waypoint), *patrol* (patrouille entre plusieurs waypoints), *loop* (déplacement en boucle) et *charge* (retour à la station de charge). La communication avec Open-RMF a été validée via les topics `/task_api_requests` et `/task_api_responses`.

3.4.6 Le Dispatcher et l'allocation des missions

Le *Dispatcher* est le composant d'Open-RMF chargé de l'allocation des missions. Il reçoit les demandes de tâches, diffuse un appel d'offres (*Bid Notice*) aux flottes disponibles, compare leurs propositions à l'aide d'un *Evaluator* et sélectionne la flotte retenue, à laquelle il transmet la mission. Le Dispatcher ne commande pas les robots et ne calcule pas lui-même le coût d'exécution : les estimations de temps et de distance sont produites localement par le *task planner* de chaque flotte. Il n'intervient par ailleurs que pour les tâches dites de *dispatch*, pour lesquelles aucun robot n'est imposé ; une tâche ciblant un robot précis est transmise directement à la flotte concernée.

L'installation CNIMI comportant deux flottes, l'appel d'offres met `mir_fleet` et `omron_fleet` en concurrence. Le déroulement d'une allocation est le suivant :

1. Le client publie une demande de mission sur `/task_api_requests` ; le Dispatcher crée une tâche interne.
2. Le Dispatcher diffuse un *Bid Notice* aux deux flottes.
3. Chaque flotte évalue la mission avec son *task planner*, en tenant compte de la position de ses robots, des tâches en cours, du temps de déplacement sur le graphe et des contraintes de batterie, puis renvoie une proposition (*Bid Proposal*) indiquant son meilleur robot et le temps de fin estimé.

4. Le Dispatcher compare les propositions des deux flottes à l'aide de l'*Evaluator*, qui privilégie par défaut le temps de fin estimé le plus court.
5. Le Dispatcher transmet la mission à la flotte retenue, dont le Fleet Adapter appelle `follow_new_path()` sur le robot sélectionné ; celui-ci commence à se déplacer.

À titre d'illustration, pour une mission de déplacement vers un point donné, `mir_fleet` peut proposer le MiR le mieux placé tandis que `omron_fleet` propose l'Omron LD-90 ; le Dispatcher retient alors la proposition au temps estimé le plus faible.

3.4.7 Le Traffic Schedule et la négociation de trafic

Le *Traffic Schedule* assure la coordination spatio-temporelle des déplacements. Contrairement à un évitement d'obstacles classique, qui ne réagit qu'à ce que les capteurs détectent à l'instant présent, il raisonne à la fois dans l'espace et dans le temps : chaque robot y déclare à l'avance la trajectoire qu'il compte suivre et les instants auxquels il occupera chaque portion du parcours. Ce fonctionnement prédictif distingue une flotte coordonnée par RMF d'un ensemble de robots indépendants : les conflits sont anticipés avant de survenir, puis résolus de manière globale. Le nœud `rmf_traffic_schedule` constitue le socle de cette coordination.

Lorsqu'un Fleet Adapter reçoit une mission, il calcule le chemin correspondant sur le graphe de navigation, puis soumet cette trajectoire au Traffic Schedule avant de commander le déplacement. Le robot devient alors un *participant* enregistré, identifié par un numéro unique ; ce comportement s'observe au démarrage à travers des messages du type « *Added a robot named [MiR250] with participant ID [0]* ». La trajectoire enregistrée n'est pas une simple position, mais une séquence de segments horodatés décrivant la présence prévue du robot au fil du temps.

À chaque nouvelle trajectoire soumise, le Traffic Schedule la compare aux trajectoires déjà enregistrées. Un conflit est détecté lorsque deux robots sont prévus pour occuper la même portion de l'espace au même moment, par exemple dans un couloir emprunté en sens inverse. Il ne s'agit pas d'une collision imminente, mais d'une prédiction : le système anticipe le croisement avant que les robots ne se rapprochent. Une négociation est alors ouverte : plutôt que de laisser les robots se bloquer, le système explore différentes solutions (faire attendre un robot, le détourner, ou modifier l'ordre de passage) et retient celle qui résout le conflit au moindre coût global pour la flotte. La solution est ensuite transmise aux Fleet Adapters concernés, le robot devant céder le passage s'arrêtant temporairement ou empruntant un chemin de contournement.

Au sein de notre système, le Traffic Schedule permet ainsi aux robots des deux flottes de partager le même environnement de circulation sans interférer les uns avec les autres. En réservant les trajectoires dans l'espace et le temps et en anticipant les conflits, il assure une coordination centralisée du trafic, condition nécessaire à l'exécution simultanée de missions par plusieurs robots hétérogènes.

3.4.8 Limites identifiées

Si le système remplit les fonctions attendues, sa mise au point a également révélé plusieurs points d'amélioration, sans conséquence sur le fonctionnement validé. Le premier concerne la gestion des missions sur les robots MiR : chaque commande de navigation y crée une position et une mission temporaires qui ne sont pas supprimées par la suite, ce qui explique l'accumulation d'un grand nombre de missions au fil des essais. Le deuxième tient

au vidage complet de la file de missions des MiR à chaque nouvelle navigation, susceptible d'annuler une mission soumise simultanément par un autre acteur. Le troisième concerne le *task planner*, qui applique un profil de batterie unique aux trois robots alors que leurs caractéristiques physiques diffèrent. Ces points constituent autant de pistes d'amélioration pour une future version du système.

3.5 Jumeau numérique sous Unity

La dernière couche du système est le jumeau numérique, développé sous Unity. Il assure la supervision et la visualisation en temps réel de la flotte, ainsi qu'un moyen de commande à destination de l'opérateur. L'interface développée est présentée à la Figure 3.6.

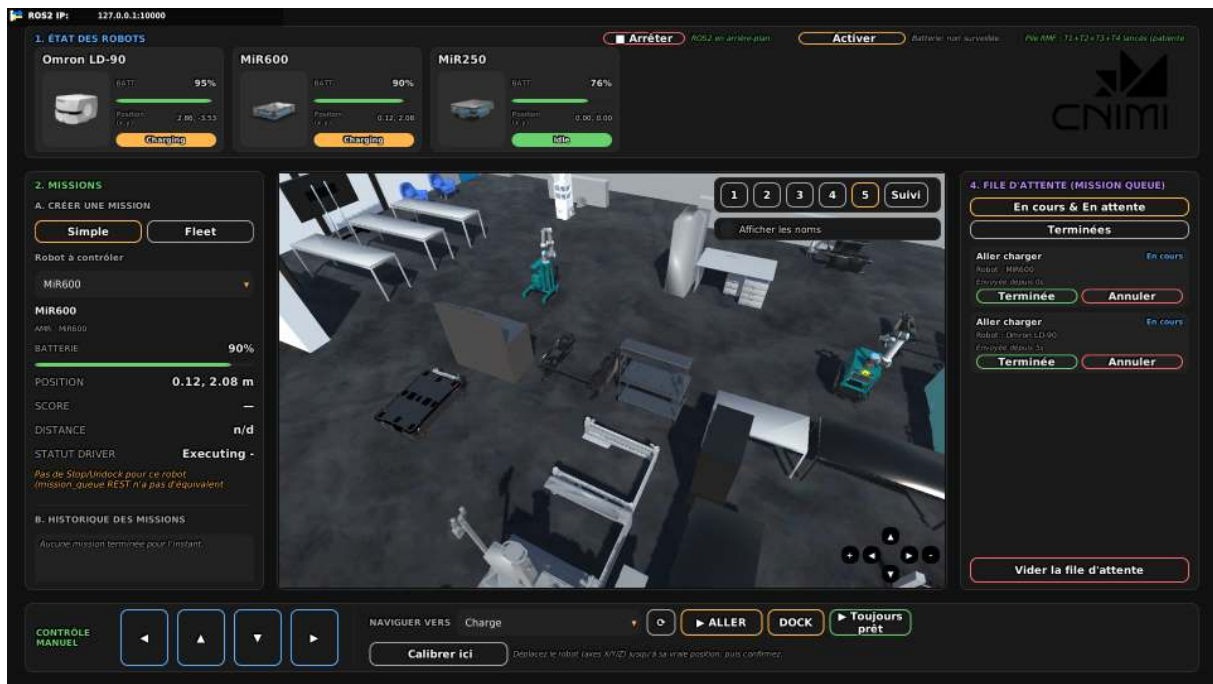


FIGURE 3.6 – Interface de commande et de supervision du jumeau numérique sous Unity

Le choix d'Unity pour cette couche découle directement des exigences du projet. La supervision d'une flotte impose un rendu temps réel fluide, une interface interactive riche et une communication bidirectionnelle avec ROS 2. Comme établi dans l'état de l'art (Chapitre 2, Tableau 2.8), les moteurs de simulation tels que Gazebo ou Isaac Sim sont avant tout orientés vers la reproduction physique et la génération de données, tandis que l'objectif recherché ici relève de la visualisation et de l'interaction plutôt que de la simulation. Unity, moteur de rendu temps réel, répond à ce besoin et présente plusieurs avantages déterminants pour ce projet :

- un **pont de communication natif avec ROS 2** via le *ROS-TCP-Connector*, qui permet de synchroniser directement la représentation virtuelle avec l'état des robots physiques ;
- la prise en charge de l'**import de modèles URDF**, facilitant l'intégration fidèle des robots dans la scène ;
- des **outils d'interface utilisateur** complets, adaptés à la conception de panneaux de supervision et de contrôle ;

- une **portabilité multiplateforme**, l’application pouvant être déployée sur différents supports (ordinateur, et à terme casque de réalité virtuelle).

Ce dernier point ouvre une perspective d’évolution importante pour le projet. Unity étant l’un des moteurs de référence pour le développement d’applications de réalité virtuelle (VR), le jumeau numérique développé pourra, dans une version future, être porté vers un environnement immersif : l’opérateur superviserait alors la flotte directement en VR, se déplaçant virtuellement dans le laboratoire CNIMI pour observer les robots et interagir avec eux. Ce prolongement, rendu possible par le choix d’Unity dès la conception, constitue l’une des principales perspectives d’extension du système.

3.5.1 Construction de l’environnement virtuel

La première étape du développement du jumeau numérique a consisté à reconstituer, dans Unity, l’environnement dans lequel évolue la flotte. L’objectif était double : offrir un contexte spatial réaliste facilitant la supervision, et garantir la cohérence entre l’espace de navigation réel des robots et sa représentation virtuelle.

Le socle de cette reconstitution est la carte au sol du laboratoire, générée par le MiR250 et partagée par l’ensemble de la flotte. L’emploi de cette même carte assure la correspondance directe entre l’espace de navigation réel, le graphe Open-RMF et la scène Unity : un point donné du laboratoire, un waypoint du graphe et sa position dans le jumeau numérique désignent ainsi le même emplacement physique.

Autour de cette carte, l’environnement tridimensionnel a été construit dans l’éditeur Unity en combinant plusieurs sources complémentaires. Une partie des objets, spécifiques au laboratoire CNIMI et absents des bibliothèques existantes, a été modélisée sur mesure à l’aide du logiciel *Blender*, puis exportée vers Unity. D’autres éléments, plus génériques (mobilier, postes de travail, machines industrielles), proviennent de bibliothèques de modèles 3D telles que l’*Unity Asset Store* et la plateforme GrabCAD. L’ensemble de ces objets a ensuite été assemblé, mis à l’échelle et positionné dans Unity de manière à reproduire fidèlement l’agencement réel de l’espace de travail.

Des repères visuels fonctionnels ont également été intégrés à la scène, notamment les zones interdites à la circulation des robots (*NO_ROBOT_ZONE*), afin que la représentation virtuelle reflète non seulement l’apparence du laboratoire mais aussi ses contraintes opérationnelles. La Figure 4.1 présente une vue de l’environnement virtuel obtenu, comparée au laboratoire réel : la disposition des postes, des machines et des équipements y est reproduite de manière cohérente, ce qui confère au jumeau numérique un rendu réaliste et directement exploitable pour la supervision.



FIGURE 3.7 – Environnement virtuel de la flotte reconstitué sous Unity (en haut) et laboratoire CNIMI réel correspondant (en bas)

3.5.2 Modélisation des robots (URDF)

Afin de représenter fidèlement les différentes plateformes robotiques dans l'environnement virtuel, des modèles tridimensionnels des robots de la flotte ont été intégrés dans Unity. Les modèles correspondent aux trois plateformes utilisées dans ce projet : le MiR250 équipé d'un bras manipulateur UR10e, le MiR600 et l'Omron LD-90.

La modélisation repose sur l'utilisation de descriptions URDF (*Unified Robot Description Format*) associées aux maillages 3D (*meshes*) des robots. Cette approche permet de conserver la structure et la hiérarchie des différents éléments composant chaque plateforme, tout en obtenant une représentation visuelle adaptée à l'environnement du jumeau numérique.

Les modèles ont ensuite été importés dans Unity à l'aide du module *URDF-Importer*. Un ajustement de la position, de l'orientation et de l'échelle a été réalisé afin d'assurer une représentation cohérente des robots dans l'environnement virtuel.

La Figure 3.8 présente les modèles tridimensionnels des différentes plateformes robotiques intégrées dans le jumeau numérique.



FIGURE 3.8 – Modèles tridimensionnels des robots intégrés dans l’environnement virtuel : MiR250 équipé du bras UR10e, MiR600 et Omron LD-90

3.5.3 Architecture de communication Unity–ROS 2

L’interface Unity repose sur **deux mécanismes de communication distincts**, qu’il importe de ne pas confondre (Figure 3.9) :

- **Échange de topics ROS 2** via le *ROS-TCP-Connector* : Unity publie et s’abonne à des topics ROS 2 à travers le pont réseau *ros_tcp_endpoint* (port TCP 10000). Ce mécanisme, utilisé par la grande majorité des fonctions de l’interface, reste opérationnel même lorsque Unity s’exécute sur une machine différente de celle qui héberge la pile ROS, dès lors qu’elles sont reliées par le réseau.
- **Lancement de processus locaux** (*System.Diagnostics.Process*) : réservé aux commandes qui démarrent ou arrêtent la pile ROS elle-même et la surveillance de batterie. Unity exécute alors directement des commandes *ros2* sur sa propre machine ; ce mécanisme suppose donc qu’Unity et la pile ROS s’exécutent sur le même hôte.

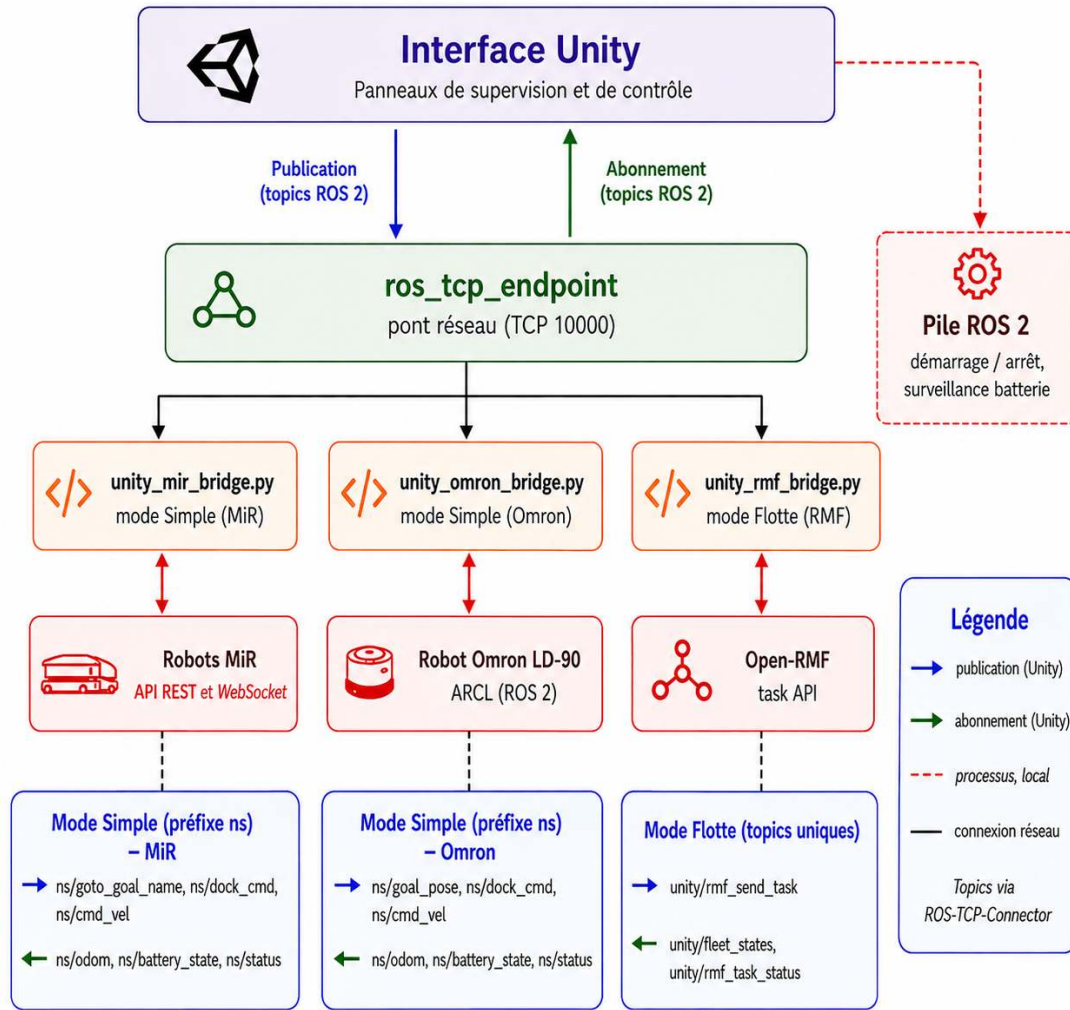


FIGURE 3.9 – Architecture de communication entre Unity et ROS 2

Comme l'illustre la Figure 3.9, l'interface est reliée aux robots par l'intermédiaire de nœuds ponts dédiés : *unity_mir_bridge.py* pour les robots MiR (API REST et WebSocket), *unity_omron_bridge.py* pour l'Omron LD-90 (protocole ARCL), et *unity_rmfm_bridge.py* pour la gestion coordonnée via Open-RMF. L'interface propose ainsi deux modes de fonctionnement complémentaires : un **mode Simple**, qui communique individuellement avec chaque robot, et un **mode Flotte**, qui s'appuie sur Open-RMF pour une gestion coordonnée.

3.5.4 Mode Simple : supervision et contrôle par robot

Le mode Simple permet à l'opérateur de superviser et de commander chaque robot individuellement. Chaque robot (MiR250, MiR600, Omron LD-90) dispose de son propre panneau, associé à un espace de noms distinct (*/mir*, */mir600*, */omron*); tous les topics de ce mode sont préfixés par le namespace du robot concerné (noté *ns* dans les tableaux qui suivent). Des nœuds ponts dédiés (*unity_mir_bridge.py* pour les robots MiR et *unity_omron_bridge.py* pour l'Omron LD-90) traduisent les commandes issues d'Unity vers l'interface native de chaque robot (API REST et WebSocket pour les MiR,

driver ROS 2 pour l’Omron) et renvoient les informations d’état vers l’interface.

Dans ce mode, la navigation ne s’effectue pas par l’envoi de coordonnées : l’opérateur sélectionne un point de destination parmi les **missions prédéfinies déjà enregistrées dans le robot**. Unity transmet alors le nom de la destination choisie, et le nœud pont déclenche la mission correspondante sur le robot (par exemple en l’ajoutant à la file de missions du MiR via son API). Les destinations disponibles sont récupérées directement depuis le robot et présentées à l’opérateur dans l’interface.

Dans le sens Unity vers robot, l’interface publie des commandes de navigation et de pilotage (Tableau 3.9). Dans le sens robot vers Unity, elle s’abonne à la télémétrie du robot pour mettre à jour l’affichage en continu (Tableau 3.10).

TABLE 3.9 – Mode Simple : principales commandes envoyées par Unity

Action	Topic	Fonction
Aller vers	ns/goto_goal_name	Lancement d’une mission prédéfinie du robot (par son nom)
Dock, Undock	ns/dock_cmd, ns/undock_cmd	Mise en charge et sortie de charge
Stop	ns/stop_cmd	Arrêt immédiat (Omron)
Jog, pas	ns/step_cmd, ns/cmd_vel	Déplacement manuel ponctuel
Rafraîchir	ns/refresh_goals_cmd	Mise à jour de la liste des destinations

TABLE 3.10 – Mode Simple : principales données reçues par Unity

Donnée	Topic	Type
Position	ns/odom	nav_msgs/Odometry
Batterie	ns/battery_state	sensor_msgs/BatteryState
Statut texte	ns/status	std_msgs/String
Score de localisation	ns/localization_score	std_msgs/Float32 (Omron)
Liste des destinations	ns/goals_list	std_msgs/String
État de mission	ns/mission_state	std_msgs/String (MiR)

3.5.5 Mode Flotte : intégration avec Open-RMF

Contrairement au mode Simple, le mode Flotte n’adresse pas les robots individuellement : il s’appuie sur Open-RMF pour une gestion coordonnée. Les échanges reposent alors sur des topics **uniques**, non préfixés par robot, gérés par un nœud pont dédié (*unity_rmf_bridge.py*) qui fait la liaison entre Unity et l’API de tâches de RMF (*/task_api_requests*, */task_api_responses*, */fleet_states*).

Lorsqu’une mission est créée dans l’interface, Unity publie une requête décrivant la tâche au format JSON (type de mission, robot ou flotte cible, points de passage, priorité). Le nœud pont la traduit en requête RMF et la soumet au Dispatcher. Les types de missions disponibles incluent le déplacement vers un point (*Go To Place*), la boucle entre deux points (*Loop*) et la mise en charge (*Charge*). En mode manuel, la flotte cible doit correspondre à celle du robot visé : *cnimi_fleet_mir* pour le MiR250 et le MiR600, et *cnimi_fleet_omron* pour l’Omron LD-90, conformément à l’enregistrement des deux flottes par le Fleet Adapter.

En retour, Unity reçoit l’état consolidé de la flotte via un topic dédié (*unity/fleet_states*), qui regroupe pour chaque robot son nom, sa tâche courante, son niveau de batterie, son mode et sa position. Ces données alimentent l’affichage temps réel de l’ensemble de la flotte sur la carte et dans le panneau de supervision.

3.5.6 Surveillance automatique de la batterie

Un module de surveillance de la batterie complète le système afin de garantir l’autonomie continue de la flotte. Ce module observe le niveau de charge des robots et déclenche automatiquement un retour à la station de recharge lorsqu’un robot passe sous un seuil bas (par exemple 20 %), le robot redevenant disponible une fois un seuil de reprise atteint (par exemple 40 %), sans intervention de l’opérateur ni planification préalable.

Ce module fonctionne en aval de l’orchestration : il s’abonne à l’état de la flotte publié par Open-RMF et agit directement sur le robot concerné, par une mission de recharge via l’API REST pour les robots MiR, ou par le service de *docking* du driver pour l’Omron LD-90. Il assure ainsi la disponibilité de la flotte sur de longues périodes de fonctionnement, en complément de la coordination assurée par Open-RMF.

3.5.7 Enregistrement de l’historique des missions

Afin de conserver une trace exploitable de l’activité de la flotte, un mécanisme d’enregistrement automatique des missions a été mis en place. Chaque mission exécutée, quel que soit le mode (Simple ou Flotte) et le robot concerné, est consignée dans un fichier permanent au format CSV (*mission_history.csv*). Une nouvelle ligne est ajoutée à la fin de chaque mission, sans que le fichier ne soit jamais réécrit, ce qui constitue un journal historique consultable a posteriori.

Pour chaque mission, l’enregistrement comporte l’horodatage de fin, le robot, le type de mission, la destination, les niveaux de batterie avant et après, la variation de batterie, la durée réelle et l’identifiant de tâche (Tableau 3.11). Le principe est identique dans tous les cas : au démarrage d’une mission, le système relève l’heure et la batterie ; à la fin, il relève de nouveau la batterie, calcule la durée et écrit la ligne. Cette logique est implémentée côté ROS 2 et devient active dès le démarrage de la pile, sans intervention de l’opérateur.

TABLE 3.11 – Champs enregistrés pour chaque mission

Champ	Description
date_heure	Horodatage de fin de mission
robot	MiR250, MiR600 ou Omron LD-90
type_mission	Type de mission (Go To Place, Loop, Charge, ou variantes du mode Simple)
destination	Point cible, ou trajet départ-arrivée pour une boucle
batterie_avant, batterie_apres	Niveau de batterie au début et à la fin (%)
delta_batterie	Variation de batterie (négatif = consommation)
duree_s	Durée réelle de la mission (s)
task_id	Identifiant de tâche RMF ou de mission MiR (vide pour l’Omron en mode Simple)

La détection de la fin de mission diffère selon le contexte, car chaque système sous-jacent expose des signaux différents. En mode Flotte, elle s’appuie sur le champ *task_id* publié par RMF pour chaque robot : le passage de ce champ de vide à rempli marque le début de la mission, et son retour à vide en marque la fin, une information fiable puisque pilotée directement par RMF. En mode Simple pour les robots MiR, la fin est détectée via l’identifiant de la file de missions du robot, interrogé périodiquement jusqu’à l’état *Done*, *Aborted* ou *Failed*. Pour l’Omron LD-90 en mode Simple, en l’absence d’un tel signal, la fin est déduite de l’arrêt physique du robot : sa vitesse est surveillée en continu et la mission est considérée terminée après deux secondes d’immobilité consécutives.

3.5.8 Gestion du système et intégration de bout en bout

Au-delà du contrôle des robots, l’interface intègre des commandes de gestion de la pile logicielle ROS 2 : démarrage et arrêt de l’ensemble des nœuds du système ainsi qu’activation de la surveillance de la batterie. Contrairement aux fonctions de supervision et de commande, ces opérations ne transitent pas par des topics ROS 2 : elles s’appuient sur le mécanisme de lancement de processus locaux et exécutent directement les commandes *ros2* correspondantes sur la machine hôte. Un unique ensemble de boutons permet ainsi de déployer ou d’interrompre la totalité de l’architecture (drivers, orchestration Open-RMF, nœuds ponts et surveillance de batterie) sans recourir à la ligne de commande.

L’articulation de l’ensemble des composants se vérifie sur le trajet complet d’une commande de navigation en mode Simple, qui traverse successivement toutes les couches du système. L’opérateur sélectionne d’abord une destination dans l’interface Unity, qui publie le nom du point cible sur le topic associé, acheminé jusqu’à ROS 2 par le *ROS-TCP-Connector*. Le nœud pont traduit ensuite ce nom en une mission concrète et la transmet à l’interface native du robot. Le robot exécute alors la mission à l’aide de son système de navigation embarqué, tandis que le nœud pont interroge périodiquement son état d’avancement et le retransmet vers Unity. En parallèle, la position et le niveau de batterie du robot sont diffusés en continu afin de maintenir l’affichage à jour en temps réel.

Ce trajet illustre la cohérence de l'architecture développée : depuis l'action de l'opérateur jusqu'à l'exécution physique par le robot, chaque couche (interface, communication, adaptation et navigation) remplit un rôle défini et s'articule avec les autres. Le jumeau numérique constitue ainsi la couche de supervision unifiée du système, reliant l'opérateur à une flotte hétérogène à travers une architecture commune, cohérente et interopérable.

Conclusion

Ce chapitre a présenté la conception et l'implémentation de la solution proposée pour la gestion et la supervision d'une flotte hétérogène de robots mobiles. À partir de l'analyse des besoins et de l'architecture globale du système, chaque couche a été développée et intégrée de manière progressive.

Une couche d'intégration a d'abord été mise en place pour unifier l'accès aux robots malgré leurs interfaces propriétaires distinctes, à travers des drivers dédiés traduisant les protocoles REST et ARCL vers un modèle ROS 2 commun. Cette couche a ensuite servi de socle à l'orchestration de la flotte par Open-RMF, qui assure l'allocation des missions, la coordination du trafic et la gestion des deux flottes au sein d'un référentiel spatial commun. Enfin, un jumeau numérique développé sous Unity a été réalisé pour offrir une supervision et une visualisation en temps réel de l'ensemble de la flotte, ainsi qu'un moyen de commande à destination de l'opérateur.

L'ensemble de ces contributions répond aux objectifs spécifiques définis au premier chapitre : la couche d'intégration et le référentiel commun (OS1 et OS2), l'orchestration par Open-RMF et le client de missions (OS3 et OS4), ainsi que le jumeau numérique de supervision (OS5). Le système obtenu constitue une architecture cohérente et interopérable, capable de coordonner des robots de constructeurs différents dans un même environnement partagé. La validation expérimentale de cette architecture, à travers des scénarios de mission en modes Simple et Flotte, fait l'objet du chapitre suivant.

Chapitre 4 : Tests et validation expérimentale

Chapitre 4 : Tests et validation expérimentale

Introduction

Après avoir présenté, dans le chapitre précédent, la conception et l'implémentation de l'architecture de gestion et de supervision de la flotte, ce chapitre est consacré à sa validation expérimentale sur les plateformes robotiques réelles du laboratoire CNIMI.

L'objectif de cette campagne d'essais est de vérifier que les différentes fonctions développées répondent aux besoins et aux objectifs définis au Chapitre 1. La validation porte notamment sur l'intégration des robots hétérogènes, l'exécution des missions, la coordination automatique de la flotte par Open-RMF, la gestion des conflits de trafic ainsi que la synchronisation des robots physiques avec leur représentation virtuelle sous Unity.

Dans un premier temps, l'environnement matériel et logiciel ainsi que la méthodologie expérimentale sont présentés. Les essais sont ensuite analysés selon les deux modes de fonctionnement du système. Le **mode Simple** permet de valider le contrôle et la supervision individuelle de chaque plateforme, tandis que le **mode Flotte** permet d'évaluer l'allocation automatique des missions et la coordination de la flotte par Open-RMF.

Enfin, des essais complémentaires sont consacrés à la robustesse de l'architecture, à la gestion des conflits de trajectoire et à la fidélité de la synchronisation avec le jumeau numérique. Les résultats obtenus sont ensuite discutés au regard des objectifs du projet et des limites de la campagne expérimentale.

4.1 Environnement et protocole de test

4.1.1 Environnement expérimental

Les essais expérimentaux ont été réalisés sur la flotte robotique réelle du laboratoire CNIMI, composée de trois plateformes mobiles autonomes : le MiR250, le MiR600 et l'Omron LD-90. La diversité de ces plateformes constitue un élément important de la campagne de validation, puisqu'elles reposent sur des interfaces et des mécanismes de communication différents, intégrés dans une architecture commune de supervision et de coordination.

L'ensemble de la chaîne logicielle était exécuté sur une même machine hôte. Cette dernière regroupait la pile ROS 2 Jazzy, les drivers dédiés aux différentes plateformes, les composants d'orchestration Open-RMF, les nœuds ponts assurant les échanges avec les robots ainsi que les services de surveillance de la batterie. L'interface de supervision développée sous Unity était également exécutée sur cette même machine.

Cette configuration permet de mettre en œuvre les deux mécanismes de communication décrits au chapitre précédent. Les échanges de données entre Unity et ROS 2 reposent sur le ROS-TCP-Connector, tandis que les opérations de gestion de la pile logicielle, notamment son démarrage et son arrêt, s'appuient sur le lancement de processus locaux.

Le Tableau 4.1 récapitule les principaux éléments matériels et logiciels mobilisés lors de la campagne expérimentale.

TABLE 4.1 – Environnement matériel et logiciel des essais

Élément	Description
Machine hôte	Ordinateur unique hébergeant la pile ROS 2 et l'interface Unity
Système ROS	ROS 2 Jazzy
Robots testés	MiR250, MiR600 et Omron LD-90 (plateformes réelles)
Orchestration	Open-RMF avec deux flottes : <code>cnimi_fleet_mir</code> et <code>cnimi_fleet_omron</code>
Supervision	Jumeau numérique Unity utilisant le ROS-TCP-Connector (port 10000)
Journalisation	Enregistrement automatique des informations relatives aux missions dans un fichier CSV

4.1.2 Protocole expérimental

Afin d'assurer une procédure de validation cohérente et reproductible, les différents essais ont été réalisés selon une séquence commune.

Dans un premier temps, la pile logicielle complète est démarrée depuis l'interface de supervision. Cette opération permet de lancer les différents composants nécessaires au fonctionnement de l'architecture, notamment les drivers des robots, les nœuds ponts, les composants Open-RMF et les services de surveillance.

Une fois la pile opérationnelle, l'état initial des robots est vérifié depuis l'interface Unity. Cette vérification porte notamment sur la présence des plateformes dans le système, leur position, leur niveau de batterie et leur état de fonctionnement.

La mission de test est ensuite créée depuis Unity selon le mode de fonctionnement considéré. En mode Simple, la mission est adressée directement à un robot déterminé. En mode Flotte, elle est soumise à Open-RMF afin que le Dispatcher puisse déterminer automatiquement la plateforme chargée de son exécution.

Pendant l'exécution de la mission, les informations relatives au robot et à son évolution sont collectées automatiquement. Le système enregistre notamment l'horodatage, l'identifiant du robot, le type de mission, la destination, les niveaux de batterie avant et après l'exécution ainsi que la durée réelle de la mission.

À la fin de chaque essai, le comportement observé est comparé au résultat attendu pour le scénario considéré. Les données enregistrées sont ensuite utilisées pour l'analyse des performances présentée dans les sections suivantes.

4.2 Validation en mode Simple

Le mode Simple constitue le premier niveau de validation expérimentale de l'architecture développée. Il permet à l'opérateur de contrôler et de superviser individuellement chacune des plateformes de la flotte, sans faire intervenir le mécanisme d'allocation automatique des missions d'Open-RMF.

La campagne d'essais a consisté à commander séparément les trois robots de la flotte réelle du laboratoire CNIMI, à savoir le MiR250, le MiR600 et l'Omron LD-90, depuis

l'interface de supervision développée sous Unity. L'objectif est de vérifier, d'une part, que les commandes émises depuis l'interface sont correctement transmises au robot concerné et, d'autre part, que les informations provenant du robot sont correctement remontées et représentées dans l'environnement virtuel.

Les fonctions évaluées comprennent la navigation vers une destination, la mise en charge et la sortie de charge, l'arrêt du robot, le déplacement manuel, la calibration de la position ainsi que la visualisation en temps réel de l'état des plateformes. Le Tableau 4.2 présente la synthèse des fonctions évaluées et des critères retenus pour leur validation.

TABLE 4.2 – Fonctions évaluées lors de la validation du mode Simple

Fonction	Robots concernés	Critère de validation	Résultat
Navigation vers un point (« Aller vers »)	MiR250, MiR600, Omron	Le robot atteint la destination sélectionnée depuis Unity	Validé
Mise en charge (Dock)	MiR250, MiR600, Omron	La commande de charge et de sortie est correctement exécutée	Validé
Arrêt (Stop)	MiR250, MiR600, Omron	Le robot s'arrête à la réception de la commande	Validé
Déplacement manuel	MiR250, MiR600, Omron	Le robot réagit aux commandes de déplacement émises depuis Unity	Validé
Calibration de position (« Calibrer ici »)	MiR250, MiR600, Omron	La position du robot peut être recalée depuis l'interface	Validé
Supervision temps réel	MiR250, MiR600, Omron	La position, la batterie et le statut sont correctement affichés	Validé

4.2.1 Validation des fonctions de commande

Dans le mode Simple, chaque plateforme peut être sélectionnée individuellement depuis l'interface Unity. Les commandes sont ensuite transmises au système d'intégration correspondant au robot sélectionné. Cette organisation permet de conserver les mécanismes de navigation et de communication propres à chaque plateforme tout en proposant une interface de commande commune.

Navigation vers une destination

La première fonction évaluée concerne la navigation vers une destination. Contrairement à une commande directe basée sur l'envoi de coordonnées cartésiennes, l'opérateur sélectionne une destination parmi les missions prédéfinies disponibles pour le robot.

Une fois la destination sélectionnée, Unity transmet son identifiant au nœud pont correspondant. Celui-ci déclenche alors la mission associée sur le robot concerné. Cette

approche permet de conserver le système de navigation propre à chaque constructeur tout en masquant ses particularités au niveau de l'interface de supervision.

Les essais réalisés sur le MiR250, le MiR600 et l'Omron LD-90 ont montré que la sélection d'une destination depuis Unity entraîne correctement l'exécution de la mission correspondante sur le robot ciblé. Le robot se déplace vers la destination sélectionnée et son évolution peut être suivie depuis l'environnement virtuel.

Cette validation est particulièrement importante dans le contexte d'une flotte hétérogène, puisque la même interface permet de commander plusieurs plateformes malgré leurs différences au niveau des interfaces de communication et des mécanismes de navigation.

Mise en charge

La fonction de mise en charge a été évaluée sur les trois plateformes. Cependant, son implémentation diffère selon le constructeur en raison des mécanismes de commande propres à chaque robot.

Pour les robots MiR250 et MiR600, la mise en charge est réalisée par le lancement d'une mission dédiée à la charge. Lorsque l'opérateur sélectionne la commande correspondante depuis l'interface Unity, le nœud d'intégration MiR déclenche la mission de charge à travers l'interface de commande du robot. Le robot exécute alors sa procédure de retour vers la station de charge conformément à son système de navigation embarqué.

Pour l'Omron LD-90, la mise en charge repose sur une fonction *Dock* dédiée. La commande *Dock* est transmise au robot par l'intermédiaire de son interface d'intégration et déclenche directement la procédure d'accostage à la station de charge.

Ainsi, bien que l'action demandée par l'opérateur soit identique depuis l'interface Unity, son traitement diffère selon la plateforme. Cette différence est masquée au niveau de l'interface de supervision grâce à la couche d'intégration développée pour chaque type de robot.

Les essais réalisés ont permis de vérifier que la commande de mise en charge produit le comportement attendu sur les trois plateformes. Cette validation montre que l'architecture permet de proposer une fonction commune de supervision tout en conservant les mécanismes de commande spécifiques à chaque constructeur.

Commande d'arrêt

La fonction *Stop* permet d'interrompre le déplacement du robot sélectionné. Cette fonction constitue un élément essentiel de la supervision puisqu'elle permet à l'opérateur de réagir rapidement à une situation nécessitant l'arrêt de la plateforme.

Lors des essais, la commande d'arrêt a été envoyée depuis l'interface Unity pendant le fonctionnement du robot. L'arrêt observé sur les plateformes physiques correspond au comportement attendu, ce qui confirme la bonne transmission de cette commande à travers la chaîne d'intégration.

Déplacement manuel

Le déplacement manuel permet à l'opérateur de piloter directement le robot depuis l'interface de supervision Unity à l'aide de quatre flèches de commande directionnelles. Ces commandes permettent de contrôler les déplacements élémentaires du robot sans lancer de mission de navigation prédéfinie.

Les quatre commandes correspondent aux directions suivantes :

- $\uparrow$: déplacement vers l'avant ;
- $\downarrow$: déplacement vers l'arrière ;
- $\leftarrow$: déplacement vers la gauche ;
- $\rightarrow$: déplacement vers la droite.

Lorsqu'une flèche est actionnée, Unity transmet la commande directionnelle correspondante au système d'intégration du robot sélectionné. La commande est ensuite traduite selon le mécanisme de communication propre à la plateforme concernée et transmise au robot réel.

Cette fonction est particulièrement utile lors des opérations de positionnement, de vérification du comportement du robot ou de mise en place de la plateforme dans l'environnement. Elle permet également de réaliser des ajustements ponctuels de la position sans avoir à créer ou lancer une mission de navigation complète.

Les essais réalisés sur le MiR250, le MiR600 et l'Omron LD-90 ont permis de vérifier le fonctionnement des quatre directions de déplacement. Les robots ont réagi aux commandes émises depuis l'interface Unity, avec une correspondance entre la direction sélectionnée dans l'interface et le mouvement observé sur la plateforme physique. Ces résultats confirment le bon fonctionnement de la chaîne de commande manuelle entre l'interface Unity et les différentes plateformes robotiques.

Calibration de la position

La fonction de calibration permet de corriger la position associée au robot dans le référentiel utilisé par le système. Cette opération est importante pour maintenir la cohérence entre la localisation réelle de la plateforme, les données remontées par ROS 2 et sa représentation dans l'environnement Unity.

Lorsqu'une calibration est demandée depuis l'interface, la nouvelle référence de position est prise en compte par le système de supervision. Cette fonction permet notamment de limiter les écarts susceptibles d'apparaître entre la position réelle du robot et celle affichée dans le jumeau numérique.

Les essais réalisés ont confirmé le bon fonctionnement de cette fonction sur les plateformes considérées.

4.2.2 Validation de la supervision temps réel

La validation du mode Simple ne porte pas uniquement sur l'envoi des commandes. Elle concerne également la remontée des informations provenant des robots vers l'interface Unity.

Pendant les essais, plusieurs informations relatives à l'état des plateformes ont été observées depuis l'interface, notamment leur position, leur niveau de batterie et leur statut. Ces données permettent à l'opérateur de suivre l'évolution de chaque robot depuis un point de supervision centralisé.

La chaîne de communication repose sur ROS 2 pour la circulation des informations issues des robots et sur le ROS-TCP-Connector pour leur transmission vers Unity. Les informations reçues sont ensuite utilisées pour mettre à jour les différents éléments du modèle virtuel.

La synchronisation de la position constitue un élément particulièrement important. Lorsqu'un robot se déplace dans l'environnement réel, sa représentation correspondante dans Unity évolue simultanément. De la même manière, les changements d'état et les informations de batterie sont actualisés dans l'interface.

Lors des essais réalisés, aucune latence perceptible n’a été observée entre les mouvements des robots physiques et la mise à jour de leur représentation dans Unity. Cette observation confirme le bon fonctionnement de la chaîne de communication mise en place pour la supervision. L’ensemble des essais de commande réalisés en mode Simple est synthétisé dans le Tableau 4.3. Les résultats montrent que les fonctions testées ont produit le comportement attendu sur les trois plateformes.

TABLE 4.3 – Synthèse des essais de commande réalisés en mode Simple

Fonction testée	Plateformes	Critère de validation	Résultat
Navigation vers une destination	MiR250, MiR600, Omron LD-90	La destination sélectionnée depuis Unity est atteinte par le robot ciblé.	Validé
Mise en charge	MiR250, MiR600, Omron LD-90	La commande déclenche correctement la procédure de charge propre à chaque plateforme.	Validé
Commande d’arrêt	MiR250, MiR600, Omron LD-90	Le robot interrompt son déplacement après réception de la commande.	Validé
Déplacement manuel	MiR250, MiR600, Omron LD-90	Les quatre commandes directionnelles permettent les déplacements avant, arrière, gauche et droite.	Validé
Calibration de position	MiR250, MiR600, Omron LD-90	La position du robot peut être recalée depuis l’interface Unity.	Validé

4.2.3 Représentation de la flotte dans l’environnement virtuel

L’un des objectifs du mode Simple est également de fournir une représentation centralisée des différentes plateformes. La Figure 4.1 présente les trois robots dans l’environnement virtuel Unity.

Chaque modèle virtuel correspond à une plateforme physique du laboratoire. Sa position dans la scène est déterminée à partir des informations transmises par ROS 2. Cette représentation permet ainsi de suivre simultanément les différents robots depuis un même environnement.

La Figure 4.1 met également en évidence l’intérêt du jumeau numérique pour la supervision d’une flotte hétérogène. Les plateformes peuvent être représentées dans un environnement commun malgré leurs différences matérielles et logicielles.

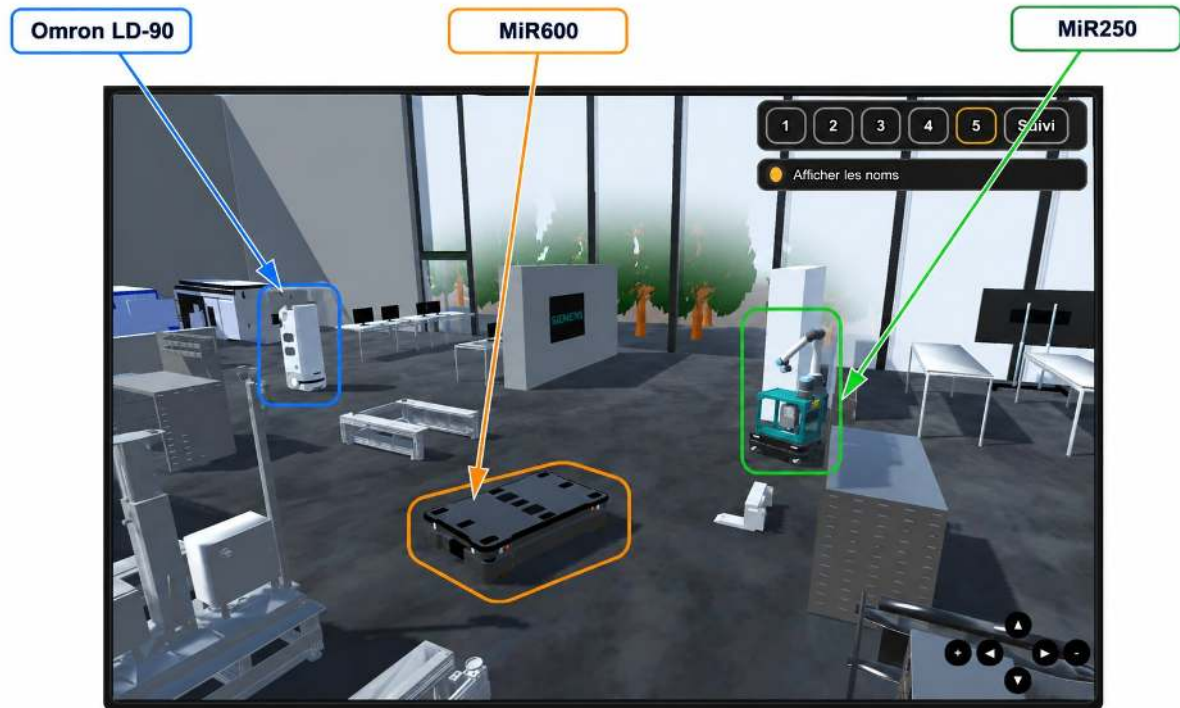


FIGURE 4.1 – Supervision simultanée des trois robots de la flotte dans l’environnement virtuel Unity

4.2.4 Analyse des résultats

Les résultats présentés dans le Tableau 4.2 montrent que les principales fonctions de commande et de supervision prévues pour le mode Simple ont produit le comportement attendu sur les trois plateformes testées.

Du point de vue de la commande, la navigation, la mise en charge, la sortie de charge, l’arrêt et le déplacement manuel peuvent être déclenchés depuis une interface commune. Le système conserve néanmoins les mécanismes spécifiques à chaque robot au niveau des couches d’intégration. Cette séparation permet de limiter l’impact des différences entre les plateformes sur l’utilisation de l’interface de supervision.

Du point de vue de la supervision, les informations provenant des robots sont remontées vers Unity et permettent de suivre leur position, leur niveau de batterie et leur état. La représentation simultanée des trois plateformes fournit ainsi à l’opérateur une vision centralisée de la flotte.

La fonction de calibration joue également un rôle important dans cette architecture. La cohérence entre les référentiels de localisation du robot réel et celui utilisé dans l’environnement virtuel conditionne directement la fidélité de la représentation du jumeau numérique. La possibilité de recalibrer la position depuis l’interface constitue donc un mécanisme utile pour maintenir cette cohérence.

4.2.5 Bilan de la validation en mode Simple

Les essais réalisés permettent de valider le fonctionnement du mode Simple sur les trois plateformes robotiques du laboratoire CNIMI. Les principales fonctions de commande

évaluées, notamment la navigation vers une destination, la mise en charge, l'arrêt, le déplacement manuel et la calibration de position, ont produit le comportement attendu lors des essais.

La validation a également porté sur la chaîne de supervision. Les informations relatives à la position, au niveau de batterie et à l'état de fonctionnement des robots sont remontées vers l'interface Unity et actualisées dans l'environnement virtuel. La représentation simultanée du MiR250, du MiR600 et de l'Omron LD-90 permet ainsi à l'opérateur de disposer d'une vision centralisée de la flotte.

Les essais mettent également en évidence l'intérêt de la couche d'intégration développée pour gérer l'hétérogénéité des plateformes. Bien que les mécanismes de commande et les interfaces de communication diffèrent selon les constructeurs, ces particularités sont prises en charge par les couches d'intégration propres à chaque robot. L'interface Unity peut ainsi proposer des fonctions communes de commande et de supervision sans imposer une modification des systèmes embarqués des plateformes.

La mise en charge constitue notamment un exemple de cette adaptation. Pour les robots MiR250 et MiR600, l'opération est réalisée par le lancement d'une mission de charge, tandis que l'Omron LD-90 dispose d'une fonction *Dock* dédiée. Cette différence de traitement est transparente pour l'opérateur au niveau de l'interface, tout en conservant les mécanismes natifs propres à chaque plateforme.

Ainsi, la validation du mode Simple constitue une première vérification de l'interopérabilité de l'architecture développée. Elle montre qu'une même interface de supervision peut être utilisée pour commander et suivre plusieurs plateformes robotiques hétérogènes tout en conservant leurs mécanismes de fonctionnement spécifiques.

Cette première étape permet ensuite d'évaluer le système à un niveau supérieur. Dans le **mode Flotte**, l'opérateur ne désigne plus nécessairement le robot chargé d'exécuter une mission. Celle-ci est soumise à Open-RMF, qui est alors chargé de déterminer la plateforme appropriée et de coordonner son exécution au sein de la flotte. La section suivante est consacrée à la validation de ce mécanisme d'allocation et de coordination.

4.3 Validation en mode Flotte

Après avoir validé le contrôle et la supervision individuelle des plateformes en mode Simple, une deuxième campagne expérimentale a été consacrée au **mode Flotte**. Cette campagne vise à évaluer la capacité de l'architecture à gérer les missions à l'échelle de la flotte et à permettre une sélection automatique de la plateforme chargée d'exécuter une mission.

Contrairement au mode Simple, dans lequel l'opérateur sélectionne directement le robot auquel une mission doit être adressée, le mode Flotte permet de soumettre une mission au système sans imposer au préalable le robot exécutant. Le Dispatcher d'Open-RMF évalue alors les différentes possibilités d'exécution à partir des informations disponibles sur les robots et de leur trajet sur le graphe de navigation.

Dans cette expérimentation, il est important de distinguer la distance géométrique entre un robot et une destination de la distance réellement parcourue pour atteindre cette destination. La distance à vol d'oiseau correspond à une mesure directe entre deux positions et ne tient pas compte de la configuration de l'environnement ni des voies de circulation.

À l'inverse, le déplacement d'un robot dans l'environnement est réalisé en suivant un chemin composé de plusieurs segments du graphe de navigation. La longueur du trajet

correspond donc à la somme des longueurs des segments qui constituent le chemin entre la position initiale du robot et la destination. Cette représentation permet de tenir compte de la structure réelle du réseau de circulation. Pour valider ce comportement, une série de destinations a été soumise au système en mode automatique. Les mêmes destinations ont également été exécutées manuellement par les robots concernés afin de disposer d'une référence expérimentale.

Le Tableau 4.4 présente, pour chaque destination, le robot sélectionné automatiquement par Open-RMF ainsi que la distance à vol d'oiseau de chaque robot à la destination, fournie à titre comparatif.

TABLE 4.4 – Robot sélectionné par Open-RMF et distances à vol d'oiseau aux destinations

Destination	Dist. MiR250	Dist. Omron	Robot choisi
L10	14,60 m	3,46 m	Omron LD-90
L20	13,17 m	3,23 m	Omron LD-90
L23	4,54 m	9,56 m	MiR250
test123	5,54 m	14,71 m	MiR250
mir_charging_station	8,19 m	8,72 m	MiR250
L50	16,28 m	19,74 m	Omron LD-90

Les temps d'exécution correspondants, mesurés en mode automatique et en mode manuel pour chaque robot, sont présentés dans le Tableau 4.5. Pour chaque destination, le temps du robot le plus rapide en mode manuel est mis en évidence.

TABLE 4.5 – Temps d'exécution en mode automatique et en mode manuel

Destination	Robot (auto)	Auto	MiR250 (man.)	Omron (man.)
L10	Omron LD-90	23,78 s	32,89 s	20,05 s
L20	Omron LD-90	27,58 s	32,79 s	24,11 s
L23	MiR250	11,21 s	8,09 s	34,07 s
test123	MiR250	39,65 s	35,56 s	42,19 s
mir_charging_station	MiR250	40,39 s	33,19 s	36,33 s
L50	Omron LD-90	75,24 s	62,21 s	72,33 s

Ces mêmes temps sont illustrés graphiquement par la Figure 4.2, qui compare visuellement le mode automatique aux modes manuels de chaque robot pour l'ensemble des destinations testées.

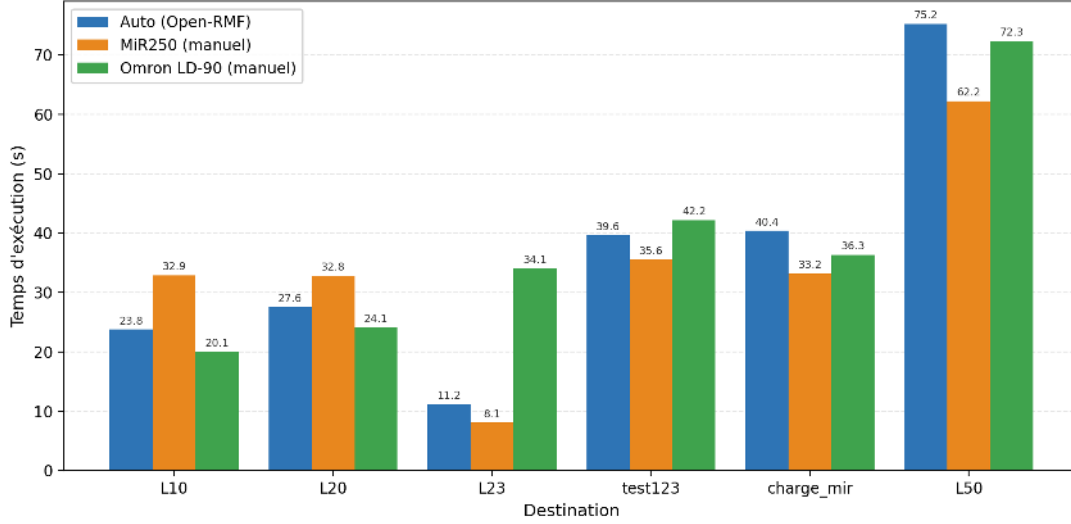


FIGURE 4.2 – Comparaison des temps d'exécution selon le mode de fonctionnement

4.3.1 Analyse de l'allocation automatique

Le Tableau 4.4 présente les distances à vol d'oiseau entre les deux plateformes et les différentes destinations, ainsi que le robot finalement sélectionné par Open-RMF.

Pour les destinations L10 et L20, l'Omron LD-90 est sélectionné. Il est également le robot présentant la plus faible distance géométrique pour ces deux destinations, avec respectivement 3,46 m et 3,23 m, contre 14,60 m et 13,17 m pour le MiR250.

Pour L23, test123 et mir_charging_station, le MiR250 est sélectionné. Dans ces trois cas, il présente également la plus faible distance à vol d'oiseau, avec respectivement 4,54 m, 5,54 m et 8,19 m.

Cette correspondance entre la distance géométrique et le robot sélectionné est observée pour la majorité des destinations testées. Cependant, elle ne doit pas être interprétée comme une règle de sélection basée directement sur la distance à vol d'oiseau. En effet, cette mesure ne représente pas nécessairement le trajet que le robot doit réellement parcourir dans l'environnement.

Le chemin considéré pour le déplacement est construit à partir du graphe de navigation. Il relie la position du robot à la destination par une succession de *waypoints* et de segments de circulation (*lanes*). La longueur du trajet peut ainsi être exprimée par :

$$L_{\text{trajet}} = \sum_{i=1}^n L_i \quad (4.1)$$

où L_i représente la longueur du i -ième segment du chemin et n le nombre total de segments constituant le trajet.

Cette approche permet donc de prendre en compte la structure du réseau de circulation plutôt qu'une simple distance directe entre deux points.

4.3.2 Comparaison des temps d'exécution

Le Tableau 4.5 permet de comparer les temps d'exécution obtenus en mode automatique avec ceux mesurés en mode manuel pour les différentes destinations.

Globalement, les temps mesurés en mode automatique restent proches de ceux obtenus en mode manuel pour le robot finalement sélectionné. La différence observée correspond principalement au délai supplémentaire introduit par l’orchestration de la mission avant son exécution effective.

En mode Flotte, la mission doit en effet être soumise au système, prise en compte par le Dispatcher, évaluée puis attribuée au robot approprié avant que celui-ci puisse commencer son déplacement. Cette succession d’étapes introduit un délai qui n’existe pas de la même manière lors d’une commande manuelle adressée directement au robot.

Les écarts observés restent toutefois limités dans les scénarios étudiés. Ils ne remettent donc pas en cause l’intérêt du mode Flotte, qui apporte en contrepartie une capacité d’allocation automatique et de coordination de plusieurs plateformes.

Il est important de distinguer ce délai d’orchestration du temps de déplacement proprement dit. Le temps supplémentaire observé ne signifie pas que le robot se déplace plus lentement ; il correspond principalement au traitement de la mission avant son exécution.

Ainsi, le Tableau 4.5 permet de vérifier que l’introduction d’Open-RMF ajoute un délai limité au lancement des missions, tout en conservant des temps d’exécution du même ordre de grandeur que ceux obtenus en commande directe.

4.3.3 Analyse du cas particulier de L50

Le point L50 constitue le cas le plus significatif de cette campagne expérimentale. La distance à vol d’oiseau entre le MiR250 et L50 est de 16,28 m, tandis que celle entre l’Omron LD-90 et L50 est de 19,74 m. Le MiR250 est donc géométriquement plus proche de la destination.

Malgré cette proximité, Open-RMF sélectionne l’Omron LD-90. Ce résultat montre que la distance directe ne suffit pas à expliquer la décision du Dispatcher.

Pour atteindre L50, chaque robot doit suivre un chemin constitué de plusieurs segments du graphe de navigation. La distance réellement associée au trajet dépend donc de la succession des segments disponibles entre la position initiale et la destination. Le nombre et la longueur de ces segments peuvent être différents selon la position initiale du robot et la structure du graphe.

Dans le cas de L50, le trajet correspondant au MiR250 est moins favorable que ne le laisse supposer sa distance à vol d’oiseau. L’Omron LD-90 est alors retenu par le système malgré une distance géométrique initialement plus importante.

Ce scénario constitue ainsi une illustration expérimentale de l’intérêt d’une allocation fondée sur le réseau de navigation. Il montre qu’une comparaison basée uniquement sur les coordonnées des robots pourrait conduire à une conclusion différente de celle obtenue en considérant le trajet réellement réalisable dans l’environnement.

4.3.4 Interprétation des résultats

Les résultats obtenus permettent de distinguer deux notions : la proximité géométrique et le coût du trajet réellement réalisable.

La distance à vol d’oiseau constitue une mesure simple et intuitive, mais elle ne tient pas compte des contraintes imposées par l’environnement. Dans un espace comportant des couloirs, des intersections et des voies de circulation prédéfinies, le chemin effectivement parcouru peut être sensiblement différent de la ligne droite reliant le robot à sa destination.

L'utilisation du graphe de navigation permet au contraire de représenter la succession des segments nécessaires au déplacement. La sélection d'un robot est ainsi réalisée en considérant un trajet compatible avec la structure de l'environnement.

Les résultats obtenus pour L10, L20, L23, `test123` et `mir_charging_station` montrent que, dans ces scénarios, le robot sélectionné par Open-RMF correspond également au robot présentant la plus faible distance à vol d'oiseau. Cette correspondance reste toutefois une observation expérimentale et ne permet pas, à elle seule, d'établir que la distance géométrique constitue le critère de sélection. Le scénario L50 met en évidence cette distinction : bien que le MiR250 soit géométriquement plus proche de la destination, l'Omron LD-90 est sélectionné par le système.

La comparaison des temps montre également que les missions exécutées en mode automatique restent du même ordre de grandeur que celles exécutées manuellement par le robot finalement sélectionné. Sur les six destinations étudiées, l'écart moyen observé entre les deux modes est d'environ 4,09 s.

Ces résultats montrent donc que l'ajout de la couche d'orchestration ne modifie pas fondamentalement la durée d'exécution des missions dans les scénarios étudiés, tout en apportant une capacité supplémentaire d'allocation automatique.

4.3.5 Bilan de la validation du mode Flotte

La campagne expérimentale permet de valider le fonctionnement du mode Flotte sur les scénarios étudiés. Une mission peut être soumise à Open-RMF sans désigner directement la plateforme exécutante, puis être attribuée automatiquement à un robot de la flotte.

Les résultats montrent que la décision du système est cohérente avec le réseau de navigation utilisé pour les déplacements. La comparaison des distances à vol d'oiseau avec les robots sélectionnés permet notamment de montrer que la proximité géométrique ne constitue pas, à elle seule, une explication suffisante du choix effectué par le Dispatcher.

Le cas L50 est particulièrement représentatif de ce comportement. Bien que le MiR250 soit géométriquement plus proche de la destination, l'Omron LD-90 est sélectionné en raison des caractéristiques du trajet construit sur le graphe de navigation.

La comparaison des temps d'exécution montre par ailleurs que les missions automatiques restent proches des temps obtenus lors de l'exécution manuelle du robot sélectionné. L'écart moyen observé sur les six destinations est d'environ 4,09 s.

Ces résultats contribuent ainsi à la validation des objectifs liés à l'orchestration et à l'allocation automatique des missions. Ils montrent qu'Open-RMF permet d'ajouter une couche de gestion collective au-dessus des mécanismes de navigation propres aux différentes plateformes, tout en conservant leur fonctionnement individuel.

La validation du mode Flotte constitue enfin une étape préalable à l'évaluation de la coordination de plusieurs robots dans des situations plus complexes, notamment lorsque plusieurs plateformes doivent partager simultanément les mêmes zones de circulation.

4.4 Robustesse et essais complémentaires

Au-delà de la validation fonctionnelle des modes Simple et Flotte, des essais complémentaires ont été réalisés afin d'évaluer le comportement du système dans différentes situations d'utilisation. Ces essais portent sur la stabilité de la pile logicielle, la coordination des robots dans une zone commune de circulation, la coexistence des deux modes

d'envoi de missions ainsi que la synchronisation entre les robots physiques et leur représentation virtuelle.

Ces expérimentations complètent les validations précédentes en vérifiant non seulement le fonctionnement des différentes fonctionnalités prises séparément, mais également le comportement global de l'architecture dans des situations représentatives de son utilisation.

4.4.1 Démarrage et arrêt répétés

La stabilité de l'architecture a été évaluée à travers plusieurs cycles d'arrêt et de redémarrage des différents composants du système.

Dans un premier scénario, la pile logicielle complète a été arrêtée puis redémarrée depuis l'interface Unity. Les différents composants nécessaires au fonctionnement de la flotte, notamment les drivers, l'orchestration Open-RMF, les ponts de communication avec Unity et les services de surveillance de la batterie, ont été relancés correctement. Après le redémarrage, les robots ont de nouveau été détectés par le système et réintégrés dans la flotte sans nécessiter de reconstruction manuelle de l'architecture.

L'application Unity a également été fermée puis relancée. Après le redémarrage de l'application, la communication avec la pile ROS 2 a été rétablie et les informations relatives aux robots ont de nouveau été disponibles dans l'interface de supervision.

Un troisième scénario a consisté à éteindre puis rallumer l'un des robots. Après son redémarrage, la plateforme concernée a été de nouveau détectée et prise en compte par le système, tandis que les autres robots ont continué leur fonctionnement.

Ces essais montrent que l'architecture supporte les opérations courantes d'arrêt et de redémarrage de ses différents composants sans nécessiter une reconfiguration complète du système. Ils mettent également en évidence la capacité de l'architecture à rétablir le fonctionnement de la flotte après une interruption d'un composant individuel.

4.4.2 Coordination de trafic

La capacité de coordination du trafic constitue l'un des principaux éléments différenciant le mode Flotte d'un fonctionnement indépendant des robots. Un scénario spécifique a donc été réalisé afin d'observer le comportement de deux plateformes devant emprunter une même zone de circulation.

Dans cet essai, deux robots ont reçu des missions dont les trajectoires étaient susceptibles de se rencontrer au niveau d'une même intersection. Open-RMF a pris en compte les trajectoires planifiées et a identifié la situation de conflit avant que les deux robots n'occupent simultanément la zone concernée.

Lors de l'exécution, l'un des robots a été temporairement maintenu en attente tandis que l'autre franchissait l'intersection. Une fois la zone libérée, le robot en attente a été autorisé à reprendre son déplacement.

Aucune collision ni situation de blocage n'a été observée durant le scénario testé. Ce comportement illustre le rôle du système de gestion du trafic d'Open-RMF, qui permet de coordonner l'utilisation des espaces partagés entre plusieurs plateformes.

Cet essai met ainsi en évidence une fonctionnalité qui ne peut pas être obtenue par le simple contrôle individuel des robots. Alors que le mode Simple permet de commander chaque plateforme indépendamment, le mode Flotte permet de prendre en compte

les déplacements de plusieurs robots dans un même environnement et d'organiser leur circulation.

4.4.3 Coexistence des deux modes d'envoi de missions

Un essai complémentaire a été réalisé afin de vérifier que les deux modes de fonctionnement peuvent être utilisés au sein de la même architecture sans interférence.

En mode Simple, une mission a été adressée directement à un robot déterminé. Le robot ciblé a alors exécuté la commande selon son mécanisme d'intégration propre.

En mode Flotte, une mission a été soumise à Open-RMF sans imposer directement le robot exécutant. Le Dispatcher a alors procédé à l'affectation de la mission selon le mécanisme décrit précédemment.

La coexistence de ces deux possibilités permet de répondre à des besoins différents. Le mode Simple conserve une logique de contrôle direct, notamment lorsqu'une intervention sur une plateforme particulière est nécessaire. Le mode Flotte permet quant à lui de déléguer l'affectation des missions au système d'orchestration.

Les essais réalisés montrent que les deux mécanismes peuvent fonctionner dans une même architecture et que l'opérateur peut choisir le mode adapté à la situation considérée.

4.4.4 Synchronisation en temps réel

Un dernier essai a porté sur la cohérence entre l'état du robot réel et sa représentation dans le jumeau numérique Unity.

Pendant le déplacement d'une plateforme physique, sa représentation correspondante dans l'environnement virtuel a été observée simultanément. Les informations de position transmises par ROS 2 vers Unity permettent de mettre à jour la position du modèle virtuel au cours du déplacement du robot.

Lors des essais, aucune différence de position perceptible n'a été observée entre le robot physique et sa représentation virtuelle. Les changements de direction, les déplacements et les arrêts observés sur le robot réel étaient également reproduits dans l'environnement Unity.

Cette synchronisation permet ainsi à l'opérateur de disposer d'une représentation virtuelle cohérente avec l'état observé sur les plateformes réelles. Elle confirme le rôle du jumeau numérique comme interface de supervision de la flotte.

Il convient toutefois de préciser que cette validation repose sur une observation expérimentale de la synchronisation. Aucune mesure quantitative spécifique de la latence de communication n'a été réalisée dans cette campagne.

4.5 Discussion

Les différentes campagnes expérimentales réalisées permettent d'évaluer les principales fonctionnalités de l'architecture développée. Les résultats obtenus confirment le fonctionnement du contrôle individuel, de l'orchestration des missions, de la coordination du trafic et de la supervision à travers le jumeau numérique. Plusieurs enseignements peuvent cependant être dégagés de ces essais.

4.5.1 Pertinence de l'allocation des missions

Le premier enseignement concerne le mécanisme d'allocation automatique des missions. Les essais réalisés en mode Flotte montrent que le choix du robot ne peut pas être expliqué uniquement par la distance à vol d'oiseau entre la plateforme et la destination.

Le trajet considéré est construit à partir du graphe de navigation et correspond à une succession de segments reliant les différents *waypoints*. La longueur et la configuration de ce trajet peuvent donc différer sensiblement de la distance géométrique directe.

Le cas de L50 constitue une illustration particulièrement intéressante. Bien que le MiR250 soit plus proche de la destination à vol d'oiseau, Open-RMF sélectionne l'Omron LD-90. Ce résultat montre que la configuration du chemin réellement réalisable dans l'environnement joue un rôle dans l'allocation.

La qualité de cette décision reste néanmoins directement liée à la qualité du graphe de navigation. Un graphe incomplet, mal positionné ou ne représentant pas correctement les voies de circulation peut conduire à une estimation inadaptée du trajet et donc à une affectation moins pertinente.

4.5.2 Impact de l'orchestration

La comparaison des temps présentée au Tableau 4.5 montre que les temps obtenus en mode automatique restent proches de ceux mesurés en mode manuel pour le robot finalement sélectionné.

Une différence de quelques secondes est observée entre les deux modes. Cette différence peut notamment être liée aux étapes supplémentaires introduites par l'orchestration, telles que la soumission de la mission, son traitement par le Dispatcher, son attribution au robot ainsi que les échanges entre les différents composants logiciels.

Il convient cependant de distinguer le délai d'orchestration du temps de déplacement du robot. Les mesures réalisées dans cette campagne ne permettent pas d'isoler précisément la durée de chaque étape du processus. Elles permettent néanmoins de constater que, dans les scénarios étudiés, l'ajout de la couche d'orchestration n'entraîne pas de différence importante dans la durée globale des missions.

4.5.3 Apport de la coordination multi-robots

Les essais de coordination mettent en évidence un avantage important de l'utilisation d'Open-RMF dans un environnement comportant plusieurs plateformes.

Lorsque deux robots doivent emprunter une même zone de circulation, le système peut organiser leur passage afin d'éviter une occupation simultanée de la zone conflictuelle. Dans le scénario testé, l'un des robots a été temporairement mis en attente avant de reprendre son trajet une fois l'intersection libérée.

Cette capacité est particulièrement importante dans une flotte hétérogène, car elle permet de considérer les déplacements des différentes plateformes dans un même espace partagé. Elle constitue ainsi une fonctionnalité supplémentaire par rapport au contrôle indépendant validé en mode Simple.

4.5.4 Limites de la campagne expérimentale

Malgré les résultats obtenus, la campagne expérimentale présente certaines limites.

Tout d'abord, les essais ont été réalisés sur un nombre limité de plateformes et de destinations. Les résultats permettent donc de valider le fonctionnement de l'architecture dans les scénarios étudiés, mais ne constituent pas une évaluation statistique exhaustive de toutes les situations possibles.

Ensuite, le scénario de coordination de trafic testé porte sur une situation de conflit particulière impliquant deux robots. Des scénarios plus complexes, avec plusieurs plateformes simultanément engagées dans plusieurs zones conflictuelles, permettraient d'évaluer davantage la capacité du système à gérer une flotte plus importante.

Enfin, la synchronisation entre le robot réel et Unity a été évaluée qualitativement. Une mesure expérimentale de la latence de communication, de la fréquence de mise à jour et de l'erreur de position permettrait de quantifier plus précisément les performances du jumeau numérique.

Ces limites ne remettent pas en cause les résultats obtenus, mais définissent les extensions possibles d'une campagne expérimentale plus large.

Conclusion

Ce chapitre a présenté la validation expérimentale de l'architecture développée sur la flotte réelle du laboratoire CNIMI, composée du MiR250, du MiR600 et de l'Omron LD-90.

La validation en mode Simple a permis de vérifier les principales fonctions de commande et de supervision individuelle des trois plateformes. Les commandes de navigation, de mise en charge, d'arrêt, de déplacement manuel et de calibration ont produit le comportement attendu lors des essais. La remontée des informations de position, de batterie et d'état vers l'interface Unity a également été vérifiée, permettant une supervision centralisée des différentes plateformes.

La validation en mode Flotte a ensuite permis d'évaluer le mécanisme d'allocation automatique des missions par Open-RMF. Les essais ont montré que la décision du système ne peut pas être interprétée comme une simple sélection du robot le plus proche à vol d'oiseau. Le trajet est associé au graphe de navigation et à la succession des segments permettant de relier le robot à la destination. Le cas de L50 illustre clairement cette différence, puisque l'Omron LD-90 est sélectionné alors que le MiR250 est géométriquement plus proche.

La comparaison des temps d'exécution montre par ailleurs que les missions réalisées en mode automatique restent du même ordre de grandeur que celles réalisées manuellement par le robot sélectionné. Un délai supplémentaire de quelques secondes est observé, pouvant notamment être associé aux étapes d'orchestration et de communication nécessaires à l'allocation de la mission.

Les essais complémentaires ont enfin permis de vérifier la stabilité de l'architecture lors des redémarrages, la coexistence des modes Simple et Flotte, la coordination de deux robots dans une situation de conflit de trafic ainsi que la cohérence de la représentation virtuelle dans Unity.

Dans l'ensemble, les résultats obtenus valident le fonctionnement de l'architecture proposée dans les scénarios expérimentaux étudiés. Ils montrent qu'une flotte hétérogène composée de robots de constructeurs différents peut être intégrée dans une architecture commune permettant leur commande, leur supervision et leur orchestration, tout en conservant les mécanismes spécifiques à chaque plateforme.

Ces résultats constituent une base pour les perspectives d'évolution du système, notamment l'augmentation du nombre de robots, l'enrichissement des scénarios de coordination et la quantification plus précise des performances du jumeau numérique et de l'orchestration.

Conclusion et perspectives

Conclusion et perspectives

Ce projet de fin d'études a porté sur la conception et la mise en œuvre d'une architecture de gestion, de coordination et de supervision d'une flotte hétérogène de robots mobiles autonomes au sein du laboratoire CNIMI. La coexistence de plateformes de constructeurs différents, notamment le MiR250, le MiR600 et l'Omron LD-90, implique l'utilisation d'interfaces et de mécanismes de communication propres à chaque robot. Cette hétérogénéité constituait ainsi une difficulté majeure pour leur exploitation au sein d'un environnement de supervision et de coordination commun.

L'objectif principal du projet était de proposer une architecture permettant d'intégrer ces différentes plateformes dans un même système, de faciliter leur commande et leur supervision, d'assurer l'orchestration des missions et de fournir une représentation virtuelle cohérente de leur état. Pour répondre à cette problématique, une architecture en couches a été conçue et mise en œuvre autour de ROS 2, d'Open-RMF et d'Unity.

Une première contribution concerne l'intégration des plateformes robotiques. Des couches d'intégration spécifiques ont été développées afin de traduire les interfaces et protocoles propres aux différents constructeurs vers une architecture ROS 2 commune. Cette approche permet de conserver les mécanismes natifs de chaque plateforme tout en fournissant une interface homogène pour les couches supérieures. Un référentiel spatial commun a également été mis en place afin d'assurer la cohérence des positions des différents robots dans l'environnement partagé.

Une deuxième contribution concerne l'orchestration de la flotte à travers Open-RMF. L'intégration réalisée permet de soumettre des missions à l'échelle de la flotte et de laisser le système sélectionner automatiquement la proposition présentant le temps de fin estimé le plus court. Les essais expérimentaux ont notamment montré que la décision d'allocation ne peut pas être expliquée uniquement par la distance à vol d'oiseau. Le trajet est associé au graphe de navigation et à la succession des segments permettant de relier la position du robot à la destination. Le cas de la destination L50 a notamment montré qu'un robot géométriquement plus proche peut ne pas être celui sélectionné par le système.

Une troisième contribution concerne la supervision à travers un jumeau numérique développé sous Unity. L'interface réalisée permet de représenter simultanément les différentes plateformes dans un environnement virtuel et de suivre leurs informations d'état, notamment leur position, leur niveau de batterie et leur statut. Le mode Simple fournit également à l'opérateur des fonctions communes de commande, tout en conservant les mécanismes d'intégration propres à chaque robot.

La validation expérimentale réalisée sur les plateformes réelles du laboratoire a permis de vérifier le fonctionnement de ces différentes composantes. En mode Simple, les principales fonctions de commande et de supervision ont été testées sur le MiR250, le MiR600 et l'Omron LD-90. En mode Flotte, les essais ont permis de vérifier l'allocation automatique des missions et la prise en compte du graphe de navigation. Les essais complémentaires ont également montré le bon fonctionnement de la coordination du trafic dans le scénario étudié, ainsi que la stabilité de l'architecture lors des opérations de redémarrage et la cohérence de la synchronisation entre les robots physiques et leur représentation virtuelle.

L'ensemble de ces résultats montre qu'une flotte hétérogène peut être intégrée dans une architecture commune sans supprimer les mécanismes spécifiques à chaque plateforme. L'architecture développée permet ainsi d'établir une séparation claire entre les interfaces

propres aux robots et les fonctions communes de supervision et d'orchestration. Cette approche constitue un élément essentiel pour faciliter l'évolution future du système.

Perspectives

Malgré les résultats obtenus, plusieurs perspectives peuvent être envisagées afin d'améliorer les performances, l'extensibilité et les fonctionnalités de la solution développée.

Sur le plan de l'**orchestration**, une première évolution consisterait à affiner le modèle de coût utilisé pour l'allocation des missions. Les différentes plateformes pourraient notamment disposer de profils propres tenant compte de leurs caractéristiques de déplacement, de leur niveau de batterie et de leur consommation énergétique. Une telle évolution permettrait d'obtenir une allocation davantage adaptée à l'état réel de chaque robot.

La gestion des missions sur les plateformes MiR pourrait également être améliorée. Le nettoyage automatique des missions et positions temporaires, ainsi qu'une gestion plus précise de la file de missions, permettraient de réduire les risques d'interférence entre les différents mécanismes de commande et de rendre l'intégration plus robuste lors d'une utilisation prolongée.

Sur le plan de l'**extensibilité**, l'utilisation d'Open-RMF offre une base adaptée à l'intégration de nouvelles plateformes robotiques. L'ajout d'un nouveau robot pourrait être réalisé à travers une couche d'intégration spécifique à son interface de communication, tout en conservant la logique générale d'orchestration et de supervision. Cette approche permettrait d'étendre progressivement la flotte du laboratoire à d'autres plateformes sans remettre en cause l'architecture existante.

L'extension pourrait également concerner les éléments d'infrastructure de l'environnement. Des équipements tels que des portes automatiques, des ascenseurs ou des stations de charge pourraient être intégrés à l'architecture afin d'étendre la coordination au-delà des seuls robots. Une telle évolution permettrait de tendre vers un environnement industriel davantage instrumenté et entièrement coordonné.

Sur le plan de la **supervision**, le jumeau numérique développé sous Unity ouvre également la possibilité d'une évolution vers la **réalité virtuelle (VR)**. Le système pourrait alors proposer une visualisation immersive de l'environnement du laboratoire, permettant à l'opérateur d'observer les robots, de suivre leurs états et d'interagir avec les différents éléments de la flotte dans un environnement virtuel plus immersif.

Enfin, une perspective à plus long terme concerne l'intégration de fonctions d'**analyse et d'aide à la décision**. Les données déjà enregistrées par le système pourraient être exploitées afin de construire des indicateurs de performance, d'analyser l'historique des missions et de détecter automatiquement certaines anomalies ou situations inhabituelles. Le jumeau numérique pourrait ainsi évoluer progressivement d'un outil principalement dédié à la visualisation et à la supervision vers un outil d'analyse et d'optimisation de l'exploitation de la flotte.

En définitive, le travail réalisé constitue une première base opérationnelle pour la gestion d'une flotte robotique hétérogène au sein du laboratoire CNIMI. Les résultats expérimentaux obtenus démontrent la faisabilité de l'approche proposée et ouvrent la voie à une extension progressive du système vers une flotte plus importante, une coordination plus complexe et une supervision davantage orientée vers l'analyse et l'aide à la décision.

Bibliographie

- [1] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihm, “Digital twin in manufacturing : A categorical literature review and classification,” *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.
- [2] Mobile Industrial Robots, *MiR250 – Autonomous Mobile Robot, Datasheet*, 2024.
- [3] Universal Robots, *UR10e – Collaborative Industrial Robot Arm, Datasheet*, 2024.
- [4] Mobile Industrial Robots, *MiR600 – Autonomous Mobile Robot, Datasheet*, 2024.
- [5] Omron, *Omron LD-90 – Mobile Robot, Datasheet*, 2024.
- [6] H. Kagermann, W. Wahlster, and J. Helbig, “Recommendations for implementing the strategic initiative INDUSTRIE 4.0,” *Final report of the Industrie 4.0 Working Group*, 2013.
- [7] P. Stączek, J. Pizoń, W. Danilczuk, and A. Gola, “A digital twin approach for the improvement of an autonomous mobile robots (AMR’s) operating environment—a case study,” *Sensors*, vol. 21, no. 23, p. 7830, 2021.
- [8] R. Valner, H. Masnavi, I. Rybalskii, R. Põlluäär, E. Kõiv, A. Aabloo, K. Kruusamäe, and A. K. Singh, “Scalable and heterogenous mobile robot fleet-based task automation in crowded hospital environments—a field test,” *Frontiers in Robotics and AI*, vol. 9, 2022.
- [9] W. Dai, U. Rai, J. Chiun, Y. Cao, and G. Sartoretti, “Heterogeneous multi-robot task allocation and scheduling via reinforcement learning,” *IEEE Robotics and Automation Letters*, vol. 10, no. 3, pp. 2654–2661, 2025.
- [10] N. van Duijkeren, L. Palmieri, R. Lange, and A. Kleiner, “An industrial perspective on multi-agent decision making for interoperable robot navigation following the VDA5050 standard,” in *Decision Making in Multi-Agent Systems Workshop at IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022. arXiv :2311.14615.
- [11] R. Gupta, T. Asbery, Z. Merchant, A. Anwar, and J. Thomason, “RobotFleet : An open-source framework for centralized multi-robot task planning,” *arXiv preprint arXiv :2510.10379*, 2025.
- [12] Comparative Study of Digital Twin Developed in Unity and Gazebo, “Comparative study of digital twin developed in unity and gazebo,” *Electronics*, vol. 14, no. 2, p. 276, 2025.